Lecture 6 (Supplement): Generative Models — Advanced Topics from Latest Courses

Stanford CS236, MIT 6.S978, MIT 6.S184, CMU 10-423 기반 심화 보충

> **참고 강의**: Stanford CS236 (Ermon), MIT 6.S978 (Kaiming He), MIT 6.S184 (Holderrieth), CMU 10-423 (Gormley)
>
> 이 문서는 강의 1~5에서 다루지 못한 핵심 주제들을 최신 대학 강의 커리큘럼 기준으로 보충한다.

목차 (Table of Contents)

- 0. 기존 강의 복습 & 위치 확인
  - 0.1 지금까지 배운 것 (강의 1~5)
  - 0.2 빠진 것은 무엇인가?
  - 0.3 선수 지식 (Prerequisites)
  - 0.4 학습 목표 (Learning Objectives)
- 1. Energy-Based Models (EBM)
  - 1.1 왜 EBM이 중요한가?
  - 1.2 에너지 함수
  - 1.3 핵심 문제: 분배 함수
  - 1.4 Score-Based Model과의 연결
  - 1.5 PyTorch 구현: Contrastive Divergence
- 2. 생성 모델 평가 지표
  - 2.1 왜 평가가 어려운가?
  - 2.2 핵심 지표
  - 2.3 정리
  - 2.4 PyTorch 구현: FID 계산
- 3. DDPM 수식 전체 유도
  - 3.1 Forward Process
  - 3.2 Reverse Process
  - 3.3 뉴럴넷은 무엇을 학습하는가?
  - 3.4 학습 목표 (Simple Loss)
  - 3.5 생성 알고리즘
  - 3.6 Score-Based Model과의 수학적 동치 관계
  - 3.7 네트워크 구조: U-Net
- 4. Diffusion Transformer (DiT)
  - 4.1 U-Net의 한계와 DiT의 등장
  - 4.2 DiT의 구조
  - 4.3 왜 DiT가 중요한가?
  - 4.4 PyTorch 구현: adaLN-Zero Block
- 5. Latent Diffusion & Stable Diffusion 상세

    - 5.1 픽셀 vs 잠재 공간 디퓨전
    - 5.2 LDM 파이프라인
    - 5.3 텍스트 조건 주입: Cross-Attention
    - 5.4 Stable Diffusion 버전별 진화
- 6. Classifier-Free Guidance (CFG)
    - 6.1 동기
    - 6.2 Classifier-Free Guidance
    - 6.3 기하학적 해석
    - 6.4 Negative Prompt
- 7. Flow Matching
    - 7.1 Diffusion의 문제점
    - 7.2 핵심 아이디어: 직선 경로
    - 7.3 수학적 프레임워크
    - 7.4 조건부 Flow Matching
    - 7.5 Flow Matching vs DDPM 비교
    - 7.6 Optimal Transport 경로
    - 7.7 Diffusion과 Flow Matching의 통합
- 8. SDE/ODE 통합 프레임워크
    - 8.1 Yang Song의 통합
    - 8.2 확률 흐름 ODE
    - 8.3 구체적 인스턴스
    - 8.4 DDIM은 ODE 솔버
- 9. CLIP & 멀티모달 조건부 생성
    - 9.1 CLIP
    - 9.2 CLIP이 생성 모델에서 사용되는 방식
    - 9.3 왜 T5가 CLIP을 보완하는가?
- 10. Visual Autoregressive Models (VAR, MAR)
    - 10.1 기존 Visual AR 모델의 한계
    - 10.2 VAR
    - 10.3 MAR
- 11. Consistency Models & 가속 샘플링
    - 11.1 문제: 디퓨전의 느린 생성
    - 11.2 Consistency Models
    - 11.3 기타 가속 기법
    - 11.4 PyTorch 구현: Consistency Model Inference
- 12. 비디오 생성 & Physical AI
    - 12.1 이미지 → 비디오 확장
    - 12.2 주요 비디오 생성 모델
    - 12.3 Diffusion Policy
    - 12.4 World Models
- 13. 종합 로드맵: 추천 학습 순서
    - 13.1 커리큘럼 매핑
    - 13.2 추천 학습 경로
    - 13.3 추천 학습 자료

- 부록: 핵심 수식 모음
  - A. 확률 분포 모델링의 4가지 접근
  - B. 전체 수식 총정리
  - C. 생성 모델 계보도
- 참고문헌 (References)

---

0. 기존 강의 복습 & 위치 확인

0.1 지금까지 배운 것 (강의 1~5)

강의	주제	핵심 개념
1	VAE	ELBO, KL Divergence, Reparameterization Trick
2	PixelRNN, VQ-VAE, Normalizing Flow	자기회귀 모델, 이산 잠재 공간, 가역 변환
3	GAN, DCGAN, ProGAN	Minimax 게임, CNN 기반 생성, 점진적 학습
4	CycleGAN, StyleGAN, VQ-GAN	비쌍 변환, 스타일 주입, 2-Stage 생성
5	DALL-E, Score-Based Model	dVAE 토큰화, 스코어 함수, Langevin Dynamics

> **왜 이 순서로 배웠는가?** 강의 1~5는 생성 모델의 네 가지 큰 패러다임 — (1) 잠재 변수 모델(VAE), (2) 자기회귀 모델(PixelRNN, DALL-E), (3) 암시적 모델(GAN), (4) 스코어 기반 모델 — 을 역사적 순서대로 다루었다. 각 패러다임은 이전 패러다임의 한계를 극복하려는 시도에서 탄생했다. VAE는 잠재 공간을 도입했지만 흐릿한 이미지가 문제였고, GAN은 선명한 이미지를 생성했지만 학습이 불안정했으며, Score-Based Model은 이 둘의 장점을 결합하면서도 이론적으로 더 단단한 기반을 제공한다.

0.2 빠진 것은 무엇인가?

최신 대학 강의들과 비교하면 다음이 빠져 있다:

```
[이미 배운 것]                          [아직 배우지 않은 것]
                                                |
VAE, VQ-VAE ──────────────────→  ❶ Energy-Based Models (EBM)
GAN, StyleGAN ────────────────→  ❷ 생성 모델 평가 지표 (FID, IS, LPIPS)
Score-Based (개념만) ──────────→  ❸ DDPM (수식 전체 유도)
DALL-E (자기회귀) ────────────→  ❹ Diffusion Transformer (DiT)
                                 ❺ Latent Diffusion & Stable Diffusion
                                 ❻ Classifier-Free Guidance (CFG)
                                 ❼ Flow Matching (OT 경로)
                                 ❽ SDE/ODE 통합 프레임워크
                                 ❾ CLIP & 멀티모달 조건부 생성
                                 ❿ Visual Autoregressive Models (VAR, MAR)
                                 ⓫ 비디오 생성 & Physical AI
                                 ⓬ Consistency Models & 가속 샘플링
```

> **왜 이것들이 중요한가?** 위 12개 주제는 단순히 "추가 내용"이 아니다. 현재(2024~2025) 실제로 사용되는 생성 모델(Stable Diffusion, FLUX, Sora 등)을 이해하려면 반드시 알아야 하는 핵심 구성 요소들이다. 예를 들어, Stable Diffusion 3는 ❸DDPM + ❹DiT + ❺LDM + ❻CFG + ❼Flow Matching + ❾CLIP을 모두 결합한 시스템이다. 하나라도 모르면 전체 파이프라인을 이해할 수 없다.

0.3 선수 지식 (Prerequisites)

이 보충 자료를 효과적으로 학습하기 위해 필요한 선수 지식:

영역	필수 개념	관련 강의
확률/통계	베이즈 정리, 가우시안 분포, KL Divergence, 최대우도 추정(MLE)	강의 1
딥러닝 기초	CNN, Transformer (Self-Attention, Cross-Attention), ResNet	강의 2~4
생성 모델 기초	VAE (ELBO), GAN (Minimax), Autoregressive Model	강의 1~5
미적분학	편미분, 연쇄 법칙, 기울기(gradient) 개념	강의 5 (스코어 함수)
선형대수	행렬 연산, 고유값 분해, 공분산 행렬	Section 2 (FID)

> **선수 지식이 부족하면 어떻게 되는가?** 특히 확률/통계와 미적분학이 부족한 상태에서 이 자료를 읽으면, 수식을 "외우기"만 하고 "이해"하지 못하는 상황이 발생한다. 예를 들어 DDPM의 Simple Loss가 왜 $\|\epsilon - \epsilon_\theta\|^2$인지 모른 채 코드만 따라 치면, 나중에 Flow Matching의 $\|v_\theta - (x_1 - x_0)\|^2$가 왜 비슷하면서도 다른지 구분할 수 없다. 이 자료는 매 단계에서 "왜?"를 설명하므로, 선수 지식을 탄탄히 하고 읽으면 훨씬 효과적이다.

⚠️ **중요**: 강의 5의 Score-Based Model 개념(스코어 함수, Langevin Dynamics)을 반드시 복습한 후 이 자료를 읽어야 한다. Section 1(EBM)과 Section 3(DDPM)이 강의 5에 직접 연결된다.

0.4 학습 목표 (Learning Objectives)

이 보충 자료를 학습한 후 다음을 할 수 있어야 한다:

1. **EBM과 Score-Based Model의 연결 관계**를 수식으로 설명할 수 있다
2. **FID, IS, LPIPS** 등 생성 모델 평가 지표의 수식과 의미를 이해한다
3. **DDPM의 전체 수식 유도** (Forward → Reverse → Simple Loss)를 따라갈 수 있다
4. **DiT의 adaLN-Zero 메커니즘**과 Transformer 기반 디퓨전의 장점을 설명할 수 있다
5. **Latent Diffusion의 파이프라인** (AutoEncoder + Latent Diffusion + Cross-Attention)을 이해한다
6. **CFG의 수식과 기하학적 직관**을 통해 Negative Prompt의 원리를 설명할 수 있다
7. **Flow Matching과 DDPM의 차이**를 수학적 기반(ODE vs SDE)으로 비교할 수 있다
8. **SDE/ODE 통합 프레임워크** 하에서 여러 디퓨전 모델의 위치를 파악할 수 있다
9. **VAR, MAR** 등 최신 Visual AR 모델의 아이디어를 기존 DALL-E와 비교할 수 있다
10. **Consistency Models**의 원리와 가속 샘플링 기법들을 이해한다

---

1. Energy-Based Models (EBM)

> Stanford CS236: Lecture 11~14, 4주간 집중 다룸

1.1 왜 EBM이 중요한가?

Score-Based Model을 이해하려면 EBM을 먼저 알아야 한다. 사실 Score-Based Model은 EBM의 학습 문제를 해결하려는 시도에서 나왔다.

> **왜 이런 접근이 필요한가?** 강의 1~5에서 배운 모델들(VAE, GAN, Flow)은 각각 확률 분포 $p(x)$를 모델링하는 다른 방식을 사용한다. 그런데 이 모든 접근에는 공통된 제약이 있다: 확률 분포는 반드시 $\int p(x)dx = 1$을 만족해야 한다는 것이다. VAE는 이를 위해 잠재 변수를 도입했고, Flow는 가역 변환으로 정규화를 유지했으며, GAN은 아예 확률을 명시적으로 계산하지 않는 방법을 택했다. EBM은 완전히 다른 관점을 제시한다: "정규화 조건을 포기하고, 대신 상대적인 확률 비교만 하자." 이 발상의 전환이 Score-Based Model과 DDPM으로 이어지는 핵심 열쇠이다.

⚠️ **중요**: EBM은 단순한 역사적 모델이 아니라, 현대 디퓨전 모델의 **이론적 기반**이다. 스코어 함수가 왜 Z를 우회하는지 이해하려면 EBM의 문제점을 먼저 알아야 한다.

1.2 에너지 함수

확률 분포를 에너지로 표현한다:

$$p_\theta(x) = \frac{1}{Z(\theta)} \exp(-E_\theta(x))$$

- $E_\theta(x)$: **에너지 함수** (뉴럴넷으로 모델링)
- $Z(\theta) = \int \exp(-E_\theta(x))dx$: **분배 함수** (정규화 상수)
- 에너지가 낮으면 → 확률이 높음 (물리학의 볼츠만 분포와 동일)

> **왜 "에너지"라는 비유를 사용하는가?** 이 수식은 통계역학의 볼츠만 분포에서 직접 가져온 것이다. 물리학에서 시스템은 에너지가 낮은 상태에 있을 확률이 높다 (물이 높은 곳에서 낮은 곳으로 흐르는 것과 비슷하다). 마찬가지로, EBM에서 뉴럴넷 $E_\theta(x)$는 입력 x에 "에너지 점수"를 매기고, 에너지가 낮은 x일수록 "실제 데이터와 비슷하다"고 판단한다. 예를 들어,

> MNIST 숫자 이미지에서 깨끗한 "7" 이미지는 에너지가 낮고, 무작위 노이즈 이미지는 에너지가 높다.

에너지 함수의 자유도가 핵심이다: VAE의 디코더는 $p(x|z)$를 가우시안으로 제한하고, Normalizing Flow는 가역 변환만 허용한다. 하지만 EBM의 $E_\theta(x)$는 **아무 뉴럴넷이나 사용할 수 있다** — 입력 x를 받아 스칼라 하나를 출력하기만 하면 된다. 이 자유도 덕분에 EBM은 이론적으로 어떤 확률 분포든 표현할 수 있다. 문제는 이 자유도의 대가로 $Z(\theta)$가 계산 불가능해진다는 것이다.

수식 상세 분해:

$\exp(-E_\theta(x))$를 "비정규화 확률(unnormalized probability)"이라고 부른다. 이 값은 데이터 포인트 x의 상대적 가능성(likelihood)을 나타내지만, 전체 합이 1이 되지 않는다. $Z(\theta)$로 나누어야 비로소 진정한 확률 분포가 된다:

$$\underbrace{p_\theta(x)}_{\text{정규화된 확률}} = \frac{\overbrace{\exp(-E_\theta(x))}^{\text{비정규화 확률}}}{\underbrace{Z(\theta)}_{\text{정규화 상수}}}$$

1.3 핵심 문제: 분배 함수 $Z(\theta)$의 계산 불가능

$Z(\theta)$는 모든 가능한 x에 대한 적분이므로 고차원에서 계산 불가능하다.

> **왜 $Z(\theta)$를 계산할 수 없는가? — 구체적 예시** 28×28 크기의 MNIST 이미지를 생각하자. 각 픽셀이 0~255 사이의 정수 값을 가지면, 가능한 이미지의 수는 $256^{784} \approx 10^{1891}$개이다. $Z(\theta)$를 정확히 계산하려면 이 모든 이미지에 대해 $\exp(-E_\theta(x))$를 계산하고 합산해야 한다. 우주의 원자 수가 약 10^{80}개인 것을 감안하면, 이것이 왜 "물리적으로 불가능"한지 직관적으로 이해된다. 연속 공간에서는 적분으로 바뀌는데, $\int_{\mathbb{R}^{784}} \exp(-E_\theta(x))dx$는 784차원 적분이므로 수치 적분도 사실상 불가능하다.

왜 $Z(\theta)$가 문제인가? 최대우도추정(MLE)으로 학습하려면:

$$\nabla_\theta \log p_\theta(x) = \nabla_\theta \log \exp(-E_\theta(x)) - \nabla_\theta \log Z(\theta) = -\nabla_\theta E_\theta(x) - \nabla_\theta \log Z(\theta)$$

첫 번째 항은 뉴럴넷의 역전파로 쉽게 계산된다. 하지만 두 번째 항 $\nabla_\theta \log Z(\theta)$를 풀어보면:

$$\nabla_\theta \log Z(\theta) = \frac{1}{Z(\theta)} \int \exp(-E_\theta(x)) \cdot (-\nabla_\theta E_\theta(x)) dx = -\mathbb{E}_{p_\theta(x)}[\nabla_\theta E_\theta(x)]$$

즉, **모델 분포 $p_\theta(x)$에서의 기댓값**을 계산해야 하는데, $p_\theta(x)$에서 샘플링하는 것 자체가 어렵다(MCMC가 필요).

이를 해결하기 위한 접근법들:

방법	아이디어	관련 모델
Contrastive Divergence	MCMC로 Z를 근사	RBM, Deep Boltzmann Machine
Score Matching	$\nabla_x \log p$를 학습 → Z가 사라짐	Score-Based Model (강의 5)

Noise Contrastive Estimation	노이즈와 데이터를 구분하는 분류 문제로 전환	NCE
Denoising Score Matching	노이즈 추가 후 스코어 학습 → 더 안정적	DDPM의 이론적 기반

> **각 방법의 핵심 직관**:
>
> - **Contrastive Divergence**: "완벽한 Z를 계산하지 말고, MCMC를 몇 스텝만 돌려서 근사하자." 정확도는 떨어지지만 실용적이다. Hinton이 2002년에 제안하여 RBM의 학습을 가능하게 했다.
> - **Score Matching**: "$p(x)$ 자체 대신 $\nabla_x \log p(x)$를 학습하면 Z가 상수이므로 미분에서 사라진다." 가장 우아한 해결책이며, 이것이 현대 디퓨전 모델의 이론적 토대이다.
> - **NCE**: "$p(x)$와 알려진 노이즈 분포 $q(x)$를 구분하는 이진 분류기를 학습하면, 분류기의 출력으로부터 $p(x)/q(x)$ 비율을 추정할 수 있다." Word2Vec의 Negative Sampling이 바로 NCE의 특수한 경우이다.
> - **Denoising Score Matching**: "깨끗한 데이터의 스코어를 직접 학습하는 것은 어렵지만, 노이즈가 추가된 데이터의 스코어는 학습하기 쉽다." Vincent(2011)가 이 등가성을 증명했고, 이것이 DDPM의 이론적 정당성을 제공한다.

1.4 Score-Based Model과의 연결

강의 5에서 배운 스코어 함수:

$$s(x) = \nabla_x \log p(x) = \nabla_x \left[\log \frac{1}{Z} \exp(-E_\theta(x))\right] = -\nabla_x E_\theta(x)$$

스코어 = 에너지의 음의 기울기. Z가 상수이므로 기울기 계산에서 완전히 사라진다.

단계별 유도 (왜 Z가 사라지는지 엄밀히 보자):

$$\begin{aligned} s(x) &= \nabla_x \log p_\theta(x) \\ &= \nabla_x \log \left[\frac{1}{Z(\theta)} \exp(-E_\theta(x))\right] \\ &= \nabla_x \left[\log \exp(-E_\theta(x)) - \log Z(\theta)\right] \\ &= \nabla_x \left[-E_\theta(x)\right] - \nabla_x \left[\log Z(\theta)\right] \end{aligned}$$

여기서 핵심: $Z(\theta)$는 θ에만 의존하고 x에는 의존하지 않는다 ($Z(\theta) = \int \exp(-E_\theta(x'))dx'$에서 x'은 적분 변수이므로). 따라서:

$$\nabla_x \log Z(\theta) = 0$$

$$\boxed{s(x) = -\nabla_x E_\theta(x)}$$

> **핵심 통찰**: 강의 5에서 "정규화 상수가 사라진다"고 배운 것이 바로 이것이다. EBM의 학습 난제를 우회한 것이 Score-Based Model이다.

> **이것이 왜 혁명적인가?** EBM의 근본 문제는 $Z(\theta)$였다. Score Matching은 이 문제를 "미분으로 상수를 소거"하는 단순하지만 심오한 트릭으로 해결했다. 비유하자면, 산의 절대 높이

> 를 모르더라도 "어느 방향이 오르막인지"는 발밑의 경사를 느끼는 것만으로 알 수 있다. 스코어 함수는 바로 이 "경사"에 해당하며, 절대 높이(= Z)를 모르더라도 충분한 정보를 제공한다. Langevin Dynamics는 이 경사를 따라 "내리막"으로 걸어가면 확률이 높은 지점(= 실제 데이터와 비슷한 이미지)에 도달할 수 있음을 보장한다.

📄 **관련 논문**: LeCun et al., "A Tutorial on Energy-Based Learning", 2006 — EBM의 이론적 기초를 체계적으로 정리한 튜토리얼.

1.5 PyTorch 구현: Contrastive Divergence

⚠️ **참고**: 아래는 Contrastive Divergence를 이용한 EBM 학습의 **간소화된 예시**이다. 실제 대규모 학습에는 Langevin MCMC의 안정화 기법이 추가로 필요하다.

```python
import torch
import torch.nn as nn

class SimpleEBM(nn.Module):
    """간단한 Energy-Based Model.

    에너지 함수 E_θ(x)를 MLP로 모델링한다.
    낮은 에너지 = 높은 확률.
    """
    def __init__(self, input_dim=784, hidden_dim=256):
        super().__init__()
        # 에너지 함수: 입력 x → 스칼라 에너지 값
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_dim, hidden_dim),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_dim, 1)  # 스칼라 에너지 출력
        )

    def energy(self, x):
        """에너지 E_θ(x) 계산"""
        return self.net(x).squeeze(-1)

    def forward(self, x):
        return self.energy(x)


def langevin_mcmc(model, x_init, n_steps=60, step_size=10.0, noise_scale=0.
    """Langevin MCMC로 모델 분포에서 샘플링.

    강의 5의 Langevin Dynamics 수식:
    x_{t+1} = x_t - (step_size/2) * ▽_x E_θ(x_t) + noise_scale * z_t

    여기서 -▽E = ▽log p (스코어 함수)이므로,
    에너지가 낮아지는 방향으로 이동한다.
    """
    x = x_init.clone().detach().requires_grad_(True)

    for _ in range(n_steps):
        # 에너지의 기울기 계산 (▽_x E_θ(x))
        energy = model.energy(x).sum()
        grad = torch.autograd.grad(energy, x)[0]

        # Langevin 업데이트: 에너지가 낮아지는 방향으로 이동 + 노이즈
        x = x.detach() - (step_size / 2) * grad + noise_scale * torch.randn
        x.requires_grad_(True)

    return x.detach()


def train_ebm_contrastive_divergence(model, dataloader, epochs=10, lr=1e-4)
    """Contrastive Divergence로 EBM 학습.

    핵심 아이디어:
    - 실제 데이터의 에너지는 낮추고 (p_data에 높은 확률)
```

```
    - MCMC 샘플의 에너지는 높인다 (가짜 데이터에 낮은 확률)

    Loss = E[E_θ(x_real)] - E[E_θ(x_fake)]
    """
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    # 리플레이 버퍼: 이전 MCMC 샘플을 재사용하여 수렴 속도 향상
    replay_buffer = []

    for epoch in range(epochs):
        for x_real, _ in dataloader:
            x_real = x_real.view(x_real.size(0), -1)  # 평탄화

            # --- MCMC 샘플 생성 (Negative Samples) ---
            # 95% 확률로 리플레이 버퍼에서, 5% 확률로 랜덤 노이즈에서 시작
            if len(replay_buffer) > 0 and torch.rand(1).item() > 0.05:
                idx = torch.randint(0, len(replay_buffer), (x_real.size(0),
                x_init = torch.stack([replay_buffer[i] for i in idx])
            else:
                x_init = torch.randn_like(x_real)  # 균일 노이즈에서 시작

            x_fake = langevin_mcmc(model, x_init, n_steps=60)

            # 리플레이 버퍼 업데이트
            for x in x_fake:
                replay_buffer.append(x.cpu())
                if len(replay_buffer) > 10000:
                    replay_buffer.pop(0)

            # --- Contrastive Divergence Loss ---
            energy_real = model.energy(x_real).mean()   # 실제 데이터 에너지
            energy_fake = model.energy(x_fake).mean()   # 가짜 데이터 에너지

            # L = E_real - E_fake (이 값을 최소화)
            # → E_real↓, E_fake↑ → 실제 데이터에 높은 확률
            loss = energy_real - energy_fake

            # 정규화: 에너지 값이 발산하지 않도록 제한
            reg = 0.001 * (energy_real ** 2 + energy_fake ** 2)
            loss = loss + reg

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        print(f"Epoch {epoch}: E_real={energy_real:.3f}, E_fake={energy_fak
```

Contrastive Divergence는 왜 작동하는가? — 수학적 직관 위 코드의 손실 함수 $L = E_\theta(x_{\text{real}}) - E_\theta(x_{\text{fake}})$를 최소화하면, 실제 데이터의 에너지는 낮아지고(= 높은 확률), 모델이 생성한 가짜 데이터의 에너지는 높아진다(= 낮은 확률). 이것은 사실 MLE 그래디언트의 근사이다. 앞서 보았듯이 $\nabla_\theta \log p_\theta(x_{\text{data}}) = -\nabla_\theta E_\theta(x_{\text{data}}) + \mathbb{E}_{p_\theta}[\nabla_\theta E_\theta(x)]$인데, 두 번째 항의 기댓값을 정확히 계산하는 대신 MCMC 샘플 몇 개로 근사하는 것이 바로 Contrastive Divergence이다.

> **리플레이 버퍼의 역할**: 매번 랜덤 노이즈에서 MCMC를 시작하면 수렴까지 수천 스텝이 필요하다. 이전에 생성한 샘플을 "기억"해두고 거기서 시작하면, 이미 모델 분포 근처에 있으므로 훨씬 적은 스텝으로 좋은 샘플을 얻을 수 있다. 이것은 Du & Mordatch (2019, "Implicit Generation and Modeling with Energy-Based Models")에서 체계적으로 연구되었다.

---

2. 생성 모델 평가 지표

> Stanford CS236: Lecture 15에서 별도로 다룸 / MIT 6.S978: pset에서 구현

2.1 왜 평가가 어려운가?

생성 모델의 출력은 확률적이다. "좋은 이미지"의 정의가 주관적이므로 정량적 평가가 필요하다.

> **왜 이런 접근이 필요한가?** GAN, VAE, Diffusion 등 다양한 생성 모델이 경쟁하는 상황에서, "어떤 모델이 더 좋은가?"를 객관적으로 비교할 방법이 없으면 연구 진전이 불가능하다. 인간 평가(human evaluation)는 비용이 크고 재현성이 떨어진다. 예를 들어, 100명에게 "이 이미지가 더 사실적인가?"를 물어보면 문화적 배경, 개인 취향, 평가 시점에 따라 결과가 달라진다. 따라서 자동화된 정량 지표가 필수적이다. 하지만 여기에는 근본적 딜레마가 있다: "좋은 이미지"는 다차원적 개념(선명도, 다양성, 텍스트 일치도, 사실성 등)인데, 단일 숫자로 이를 포착하는 것은 원리적으로 불가능하다.

⚠️ **중요**: 단일 지표로 생성 모델을 완전히 평가하는 것은 불가능하다. FID가 낮더라도 다양성이 부족할 수 있고(mode collapse), IS가 높더라도 실제 데이터와 동떨어진 이미지를 생성할 수 있다. 항상 **복수의 지표**를 함께 사용해야 한다.

2.2 핵심 지표

(1) Inception Score (IS)

$$\mathrm{IS} = \exp\left(\mathbb{E}_{x\sim p_g}\left[D_{KL}\left(p(y \mid x) \,\|\, p(y)\right)\right]\right)$$

- $p(y \mid x)$: 생성 이미지 x를 Inception-V3에 넣었을 때 클래스 분포
- $p(y) = \mathbb{E}_x[p(y \mid x)]$: 전체 생성 이미지의 평균 클래스 분포
- **높을수록 좋음**: 각 이미지가 뚜렷한 클래스에 속하면서(품질), 전체적으로 다양한 클래스가 나오면(다양성)

> **왜 Inception-V3를 사용하는가?** Inception-V3는 ImageNet의 1000개 클래스를 분류하도록 학습된 네트워크로, 이미지의 의미적 내용을 잘 파악한다. 핵심 가정은: "ImageNet 분류에 유용한 특징은 이미지의 '의미적 품질'을 반영한다"는 것이다. 이 가정이 완벽하지는 않지만(예: 텍스처 편향 문제), 실용적으로 충분히 좋은 근사를 제공한다.

IS의 두 가지 측면을 분리해서 이해하자:

측면 1 — 품질 (각 이미지의 $p(y|x)$가 날카로움): 생성된 이미지 하나를 Inception에 넣었을 때, 출력 분포가 특정 클래스에 몰리면(예: [0, 0, 0.99, 0.01, ...]) 그 이미지는 "뚜렷한 물체"를 담고 있다는 뜻이다. 반대로 출력이 균등하면(예: [0.001, 0.001, ...]) 이미지가 모호하거나 비현실적이라는 뜻이다.

측면 2 — 다양성 (전체 $p(y)$가 균등함): 모든 생성 이미지의 $p(y|x)$를 평균내면 $p(y)$를 얻는다. 이 분포가 균등하면 다양한 클래스의 이미지를 생성하고 있다는 뜻이다. $p(y|x)$는 날카로운데

$p(y)$는 균등하면 → KL divergence가 커지고 → IS가 높아진다.

수식의 직관적 분해:

$$D_{KL}(p(y|x)\|p(y)) = \sum_y p(y|x) \log \frac{p(y|x)}{p(y)}$$

이 값이 크려면: (1) $p(y|x)$가 특정 클래스 y^*에 집중되어야 하고 (→ 품질), (2) 그 y^*에서 $p(y^*)$가 작아야 한다 (→ 다양성, 즉 모든 이미지가 같은 클래스를 생성하지 않아야 함).

한계: 실제 데이터 분포와 비교하지 않음. 따라서 "아주 선명하지만 실제 데이터와 전혀 다른 이미지"도 높은 IS를 받을 수 있다.

📄 **원본 논문**: Salimans et al., "Improved Techniques for Training GANs", NeurIPS 2016

(2) Frechet Inception Distance (FID)

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right)$$

- (μ_r, Σ_r): **실제** 이미지 특징의 평균, 공분산 (Inception-V3 중간 레이어)
- (μ_g, Σ_g): **생성** 이미지 특징의 평균, 공분산
- **낮을수록 좋음**: 두 분포가 가까울수록 좋은 생성
- 가우시안 가정이므로 두 다변량 가우시안 사이의 Frechet (Wasserstein-2) 거리

> **왜 FID가 IS보다 좋은 지표인가?** IS의 치명적 결함은 실제 데이터를 참조하지 않는다는 것이다. 극단적으로, 모델이 1000개 클래스에서 각각 딱 하나씩의 "완벽한 템플릿 이미지"만 생성해도 IS는 최대값을 달성한다. 하지만 이 모델은 실제 데이터의 다양성을 전혀 포착하지 못한다. FID는 **실제 이미지의 특징 분포와 생성 이미지의 특징 분포를 직접 비교**하므로, 이런 문제를 피할 수 있다.

FID 수식의 단계별 이해:

Step 1: 특징 추출 — 이미지를 직접 비교하지 않고, Inception-V3의 마지막 풀링 레이어(pool3, 2048차원) 출력을 사용한다. 이 2048차원 벡터는 이미지의 "의미적 요약"이다. 왜 픽셀이 아닌 특징을 비교하는가? 픽셀 공간에서는 1픽셀 이동만으로도 거리가 크게 변하지만, 특징 공간에서는 의미적으로 비슷한 이미지가 가까이 위치한다.

Step 2: 가우시안 가정 — 실제/생성 이미지의 특징 벡터들이 각각 다변량 가우시안 $\mathcal{N}(\mu_r, \Sigma_r)$, $\mathcal{N}(\mu_g, \Sigma_g)$을 따른다고 가정한다. 이 가정은 완벽하지 않지만, 2048차원에서 중심극한정리에 의해 합리적인 근사이다.

Step 3: Frechet 거리 — 두 가우시안 사이의 Wasserstein-2 거리 (= Frechet 거리)를 계산한다. 이 거리는 두 가지 성분으로 분해된다:

$$\text{FID} = \underbrace{\|\mu_r - \mu_g\|^2}_{\text{평균의 차이 (위치)}} + \underbrace{\text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})}_{\text{공분산의 차이 (형태/분산)}}$$

- **첫째 항** $\|\mu_r - \mu_g\|^2$: 두 분포의 "중심"이 얼마나 떨어져 있는지. 생성 이미지가 전체적으로 실제 이미지와 다른 스타일이면 이 값이 크다.
- **둘째 항**: 두 분포의 "형태"가 얼마나 다른지. 행렬 제곱근 $(\Sigma_r\Sigma_g)^{1/2}$는 두 공분산 행렬이 동일할 때 최대가 되며, 이 경우 둘째 항은 0이 된다.

왜 단순한 KL divergence가 아닌 Frechet 거리를 사용하는가? KL divergence는 두 가우시안이 겹치지 않을 때 무한대로 발산할 수 있다. Frechet/Wasserstein 거리는 항상 유한한 값을 가지며, "분포를 이동시키는 데 필요한 최소 비용"이라는 직관적 해석이 가능하다.

> **현재 거의 모든 생성 모델 논문에서 사용하는 표준 지표.** ImageNet 256x256 기준: DDPM ~4.9, LDM ~3.6, DiT ~2.3, VAR ~1.7

📄 **원본 논문**: Heusel et al., "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium", NeurIPS 2017

(3) LPIPS (Learned Perceptual Image Patch Similarity)

$$\mathrm{LPIPS}(x, \hat{x}) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (\hat{\phi}_l^{hw}(x) - \hat{\phi}_l^{hw}(\hat{x}))\|_2^2$$

- 사전 학습된 네트워크(VGG, AlexNet)의 중간 특징 맵에서 거리 측정
- 인간의 지각적 유사도와 가장 잘 일치
- 주로 **이미지 복원, 편집**의 품질 평가에 사용

> **왜 픽셀 MSE 대신 LPIPS를 사용하는가?** 픽셀 단위 MSE($\|x - \hat{x}\|^2$)는 인간의 지각과 심하게 괴리된다. 예를 들어, 이미지를 1픽셀 오른쪽으로 이동하면 MSE는 크지만 인간은 거의 차이를 느끼지 못한다. 반대로, 색감만 살짝 변하면 MSE는 작지만 인간에게는 크게 다르게 보일 수 있다. LPIPS는 사전 학습된 네트워크의 중간 특징 맵(VGG의 conv1_2, conv2_2, conv3_3 등)에서 거리를 측정하므로, 저수준(에지, 텍스처)부터 고수준(의미적 구조)까지의 차이를 종합적으로 반영한다.

수식 분해:

- l: 네트워크의 레이어 인덱스 (여러 레이어에서 특징을 추출)
- $\hat{\phi}_l^{hw}(x)$: 레이어 l에서 위치 (h, w)의 채널 방향 특징 벡터 (단위 정규화 적용)
- w_l: 학습된 가중치 벡터 — 어떤 채널이 지각적 유사도에 더 중요한지를 반영
- 최종 값은 모든 레이어, 모든 공간 위치에 대한 가중 평균

📄 **원본 논문**: Zhang et al., "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric", CVPR 2018

(4) CLIP Score

$$\mathrm{CLIPScore}(t, x) = \cos(\mathrm{CLIP}_{\mathrm{text}}(t), \mathrm{CLIP}_{\mathrm{image}}(x))$$

- 텍스트-이미지 정합도를 측정
- Text-to-Image 모델에서 "텍스트 조건을 얼마나 잘 따르는가" 평가

> **왜 CLIP Score가 필요한가?** FID와 IS는 "이미지 품질"과 "데이터 분포 유사도"를 측정하지만, Text-to-Image 모델에서 가장 중요한 질문 — "생성된 이미지가 텍스트 프롬프트를 잘 반영하는가?" — 에는 답하지 못한다. 예를 들어, "빨간 자동차 위에 앉은 파란 고양이"라는 프롬프트에 대해 "파란 자동차 위에 앉은 빨간 고양이"를 생성하면, FID는 좋을 수 있지만 조건 충실도는 낮다. CLIP Score는 CLIP 모델의 공유 임베딩 공간에서 텍스트와 이미지의 코사인 유사도를 계산하여, 이 "텍스트-이미지 정합도"를 정량화한다. CLIP 자체에 대한 자세한 설명은 Section 9를 참조.

📄 **관련 논문**: Hessel et al., "CLIPScore: A Reference-free Evaluation Metric for Image Captioning", EMNLP 2021

2.3 정리

지표	측정	좋은 방향	용도
IS	품질 + 다양성	높을수록	기본 품질
FID	실제 vs 생성 분포 거리	낮을수록	표준 벤치마크
LPIPS	두 이미지의 지각적 거리	낮을수록	복원 품질
CLIP Score	텍스트-이미지 정합도	높을수록	조건부 생성

> **지표 선택 가이드**: 비조건부 이미지 생성(unconditional generation)을 평가할 때는 FID + IS를 사용하고, Text-to-Image 모델을 평가할 때는 FID + CLIP Score를 함께 사용하는 것이 표준이다. 이미지 복원이나 편집 태스크에서는 LPIPS를 추가한다. 최근에는 FID의 한계(가우시안 가정, 표본 크기 의존성)를 보완하기 위해 CMMD (CLIP Maximum Mean Discrepancy), FD-DINOv2 등의 새로운 지표도 제안되고 있다.

2.4 PyTorch 구현: FID 계산

⚠️ **참고**: 실제 FID 계산에는 최소 수천 장의 이미지가 필요하다. 아래 코드는 핵심 로직을 보여주는 교육용 구현이다.

```python
import torch
import torch.nn as nn
import numpy as np
from scipy import linalg
from torchvision.models import inception_v3
from torch.utils.data import DataLoader

class FIDCalculator:
    """Frechet Inception Distance (FID) 계산기.

    FID = ||μ_r - μ_g||² + Tr(Σ_r + Σ_g - 2(Σ_r·Σ_g)^{1/2})

    두 이미지 집합의 Inception-V3 특징 분포 사이의 거리를 측정한다.
    """

    def __init__(self, device='cuda'):
        self.device = device
        # Inception-V3 모델 로드 (사전 학습된 ImageNet 가중치)
        self.inception = inception_v3(pretrained=True, transform_input=Fals
        # 마지막 분류 레이어 제거 → 2048차원 특징 벡터 추출
        self.inception.fc = nn.Identity()
        self.inception.eval().to(device)

    @torch.no_grad()
    def extract_features(self, dataloader):
        """이미지 배치에서 Inception-V3 특징 벡터 추출.

        Inception-V3의 마지막 풀링 레이어 출력 (2048차원)을 사용한다.
        이것이 FID 수식의 특징 공간이다.
        """
        features_list = []
        for images, *_ in dataloader:
            images = images.to(self.device)
            # 이미지를 299x299로 리사이즈 (Inception 입력 크기)
            if images.shape[-1] != 299:
                images = nn.functional.interpolate(images, size=299, mode='

            # 특징 벡터 추출 (2048차원)
            features = self.inception(images)
            features_list.append(features.cpu().numpy())

        return np.concatenate(features_list, axis=0)

    def compute_statistics(self, features):
        """특징 벡터의 평균(μ)과 공분산(Σ) 계산.

        FID는 특징 분포를 다변량 가우시안으로 근사하므로,
        μ와 Σ만으로 분포를 표현한다.
        """
        mu = np.mean(features, axis=0)       # 평균 벡터 (2048,)
        sigma = np.cov(features, rowvar=False)  # 공분산 행렬 (2048, 2048)
        return mu, sigma

    def compute_fid(self, mu_r, sigma_r, mu_g, sigma_g):
        """FID 수식 직접 계산.
```

```
        FID = ||μ_r - μ_g||² + Tr(Σ_r + Σ_g - 2·(Σ_r·Σ_g)^{1/2})

        (Σ_r·Σ_g)^{1/2}는 행렬 제곱근이므로 scipy.linalg.sqrtm을 사용한다.
        """
        # 항 1: 평균 차이의 제곱 노름
        diff = mu_r - mu_g
        mean_term = np.dot(diff, diff)  # ||μ_r - μ_g||²

        # 항 2: 공분산 항 — 행렬 제곱근 계산
        # sqrtm: 행렬 제곱근 (A^{1/2} s.t. A^{1/2} · A^{1/2} = A)
        covmean, _ = linalg.sqrtm(sigma_r @ sigma_g, disp=False)

        # 수치 안정성: 허수부 제거 (부동소수점 오차로 발생 가능)
        if np.iscomplexobj(covmean):
            covmean = covmean.real

        cov_term = np.trace(sigma_r + sigma_g - 2.0 * covmean)

        fid = mean_term + cov_term
        return float(fid)

    def calculate(self, real_loader, gen_loader):
        """전체 FID 계산 파이프라인.

        1. 실제/생성 이미지에서 Inception 특징 추출
        2. 각 분포의 μ, Σ 계산
        3. FID 수식 적용
        """
        # 특징 추출
        print("실제 이미지 특징 추출 중...")
        real_features = self.extract_features(real_loader)
        print("생성 이미지 특징 추출 중...")
        gen_features = self.extract_features(gen_loader)

        # 통계량 계산
        mu_r, sigma_r = self.compute_statistics(real_features)
        mu_g, sigma_g = self.compute_statistics(gen_features)

        # FID 계산
        fid_value = self.compute_fid(mu_r, sigma_r, mu_g, sigma_g)
        print(f"FID = {fid_value:.2f}")
        return fid_value


# --- 사용 예시 ---
# calculator = FIDCalculator(device='cuda')
# fid = calculator.calculate(real_dataloader, generated_dataloader)
# print(f"FID Score: {fid:.2f}")  # 낮을수록 좋음 (DiT-XL/2: ~2.27)
```

> **FID 구현의 주요 함정들**:
>
> - **표본 크기 의존성**: FID는 이미지 수가 적으면 편향이 크다. 최소 10,000장, 이상적으로는 50,000장 이상을 사용해야 안정적이다. 논문에서 "FID@50K"라고 표기하는 이유가 이것이다.

> - **리사이징 방법**: Inception에 입력하기 위해 이미지를 299x299로 리사이징하는데, bilinear vs bicubic 보간 방법에 따라 FID가 달라질 수 있다. 공정한 비교를 위해 동일한 전처리 파이프라인을 사용해야 한다.
> - **행렬 제곱근의 수치 불안정성**: `scipy.linalg.sqrtm`은 행렬이 거의 특이(singular)할 때 허수부를 가진 결과를 반환할 수 있다. 위 코드의 `covmean.real` 처리가 이를 방지한다.

---

3. DDPM 수식 전체 유도

> MIT 6.S978 Lecture 5 (Kaiming He), MIT 6.S184 전체, Stanford CS236 Lecture 16

강의 5에서 Score-Based Model의 개념을 배웠다. DDPM은 이를 실용적으로 구현한 모델이다.

> **왜 DDPM이 혁명적이었는가?** 2020년 이전, 이미지 생성은 GAN의 독무대였다. 하지만 GAN은 학습 불안정(mode collapse, training divergence)이라는 근본적 문제를 안고 있었다. DDPM(Ho et al., 2020)은 완전히 다른 접근 — "이미지에 점진적으로 노이즈를 추가하는 과정을 뒤집자" — 을 통해, GAN 없이도 고품질 이미지를 생성할 수 있음을 보여주었다. 핵심 장점은: (1) 학습이 안정적 (단순 MSE loss), (2) mode collapse가 없음 (확률적 샘플링), (3) 이론적 기반이 탄탄 (variational inference). DDPM이 이미지 생성의 패러다임을 GAN에서 Diffusion으로 전환시킨 결정적 논문이다.

📄 **핵심 논문**: Ho et al., "Denoising Diffusion Probabilistic Models", NeurIPS 2020 — 현대 디퓨전 모델의 시작점.

3.1 Forward Process (노이즈 추가)

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I)$$

- β_t: 타임스텝 t에서의 노이즈 스케줄 ($\beta_1 < \beta_2 < \cdots < \beta_T$, 보통 $T = 1000$)
- 매 스텝마다 조금씩 가우시안 노이즈를 추가

> **왜 이런 형태의 노이즈 추가인가?** $q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$는 두 가지 일을 동시에 한다: (1) 이전 이미지 x_{t-1}을 $\sqrt{1-\beta_t}$로 축소 (신호 감쇠), (2) 분산 β_t의 가우시안 노이즈를 추가. $\sqrt{1-\beta_t}$로 축소하는 이유는 분산을 일정하게 유지하기 위해서이다. 만약 축소 없이 노이즈만 추가하면, 분산이 스텝마다 증가하여 x_T가 발산한다. $\sqrt{1-\beta_t}$ 축소와 β_t 노이즈 추가를 결합하면, 각 스텝에서 분산이 보존되고 (variance preserving), $t \to \infty$에서 $x_T \to \mathcal{N}(0, I)$로 수렴한다.
>
> **β_t가 점점 커지는 이유**: 초기 스텝에서는 이미지 구조가 많이 남아 있으므로 소량의 노이즈로도 충분하다. 후반 스텝에서는 이미 노이즈가 많이 섞여 있으므로, 더 많은 노이즈를 추가해야 최종적으로 순수 가우시안에 도달한다. DDPM 원 논문에서는 $\beta_1 = 10^{-4}$에서 $\beta_T = 0.02$까지 선형으로 증가시켰다.

폐쇄형(Closed-Form) 유도

$\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ 라 하면:

$$\boxed{q(x_t \mid x_0) = \mathcal{N}(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I)}$$

유도 과정 (왜 이렇게 되는가):

Step 1: $t = 1$일 때, Forward 과정의 정의에 따라:

$$x_1 = \sqrt{\alpha_1}\, x_0 + \sqrt{1-\alpha_1}\, \epsilon_1, \quad \epsilon_1 \sim \mathcal{N}(0, I)$$

여기서 $\sqrt{\alpha_1} = \sqrt{1-\beta_1}$이고, $\sqrt{1-\alpha_1} = \sqrt{\beta_1}$이다. 가우시안 $\mathcal{N}(x; \mu, \sigma^2 I)$에서 샘플링하는 것은 $x = \mu + \sigma\epsilon$ ($\epsilon \sim \mathcal{N}(0, I)$)과 같다 (이것이 강의 1의 Reparameterization Trick이다).

Step 2: $t = 2$일 때:

$$x_2 = \sqrt{\alpha_2}\, x_1 + \sqrt{1-\alpha_2}\, \epsilon_2$$

x_1을 대입하면:

$$\begin{aligned} x_2 &= \sqrt{\alpha_2}(\sqrt{\alpha_1}\, x_0 + \sqrt{1-\alpha_1}\, \epsilon_1) + \sqrt{1-\alpha_2}\, \epsilon_2 \\ &= \sqrt{\alpha_2\alpha_1}\, x_0 + \sqrt{\alpha_2(1-\alpha_1)}\, \epsilon_1 + \sqrt{1-\alpha_2}\, \epsilon_2 \end{aligned}$$

Step 3: 두 가우시안 노이즈의 합의 분산을 계산한다. 독립 가우시안의 핵심 성질: $a\epsilon_1 + b\epsilon_2$에서 $\epsilon_1, \epsilon_2 \sim \mathcal{N}(0, I)$이 독립이면, $a\epsilon_1 + b\epsilon_2 \sim \mathcal{N}(0, (a^2+b^2)I)$이다.

따라서 노이즈 항의 분산: $\alpha_2(1-\alpha_1) + (1-\alpha_2) = \alpha_2 - \alpha_2\alpha_1 + 1 - \alpha_2 = 1 - \alpha_1\alpha_2 = 1 - \bar{\alpha}_2$

$$x_2 = \sqrt{\bar{\alpha}_2}\, x_0 + \sqrt{1-\bar{\alpha}_2}\, \bar{\epsilon}, \quad \bar{\epsilon} \sim \mathcal{N}(0, I)$$

Step 4: 이를 일반화하면 (수학적 귀납법으로 증명 가능):

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

> **$\bar{\alpha}_t$의 의미를 직관적으로 이해하자**: $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$는 원래 신호 x_0가 얼마나 남아있는지를 나타낸다. $t = 0$이면 $\bar{\alpha}_0 = 1$ (100% 원본), $t = T$이면 $\bar{\alpha}_T \approx 0$ (거의 순수 노이즈). 위 수식에서 x_t는 원본 이미지 x_0와 순수 노이즈 ϵ의 **가중 합**이다: 원본의 비율은 $\sqrt{\bar{\alpha}_t}$, 노이즈의 비율은 $\sqrt{1-\bar{\alpha}_t}$. $\sqrt{\bar{\alpha}_t}^2 + \sqrt{1-\bar{\alpha}_t}^2 = 1$이므로, 총 분산이 보존됨을 확인할 수 있다.

> **실용적 의미**: 임의의 t에서 노이즈 이미지를 **한 번에** 만들 수 있다. 1000번 반복할 필요 없음. 이것이 학습을 효율적으로 만드는 핵심이다 — 학습 시 매번 t를 랜덤 샘플링하고, 해당 노이즈 레벨의 x_t를 즉시 생성할 수 있다.

3.2 Reverse Process (노이즈 제거 = 생성)

역과정의 진짜 분포 $q(x_{t-1} \mid x_t)$는 계산 불가능하지만, x_0를 조건으로 하면 가능하다:

> **왜 $q(x_{t-1}|x_t)$가 계산 불가능한가?** 베이즈 정리로 $q(x_{t-1}|x_t) = q(x_t|x_{t-1})q(x_{t-1})/q(x_t)$인데, 분모 $q(x_t) = \int q(x_t|x_0)q(x_0)dx_0$은 전체 데이터 분포에 대한 적분이므로 계산 불가능하다 (EBM의 $Z(\theta)$와 같은 문제!). 하지만 x_0를 **알고 있다고 가정하면**, 조건부 분포 $q(x_{t-1}|x_t, x_0)$는 세 가우시안의 곱이므로 닫힌 형태로 계산 가능하다.

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1};\, \tilde{\mu}_t(x_t, x_0),\, \tilde{\beta}_t I)$$

여기서:

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$$

$$\tilde{\beta}_t = \frac{(1-\bar{\alpha}_{t-1})}{(1-\bar{\alpha}_t)}\beta_t$$

> **이 수식은 어디서 나오는가? — 베이즈 정리를 이용한 유도**
>
> $q(x_{t-1}|x_t, x_0) \propto q(x_t|x_{t-1}) \cdot q(x_{t-1}|x_0)$
>
> 두 항 모두 가우시안이다:
>
> - $q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, \beta_t I)$
> - $q(x_{t-1}|x_0) = \mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}x_0, (1-\bar{\alpha}_{t-1})I)$
>
> 두 가우시안의 곱도 가우시안이다 (completing the square). 지수부를 전개하면:
>
> $$\exp\left(-\frac{1}{2}\left[\frac{(x_t - \sqrt{\alpha_t}x_{t-1})^2}{\beta_t} + \frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}x_0)^2}{1-\bar{\alpha}_{t-1}}\right]\right)$$
>
> x_{t-1}에 대한 이차식으로 정리하면 (completing the square):
>
> $$\propto \exp\left(-\frac{(x_{t-1} - \tilde{\mu}_t)^2}{2\tilde{\beta}_t}\right)$$
>
> 이차 항의 계수를 비교하여 $\tilde{\beta}_t$를 구하고, 일차 항에서 $\tilde{\mu}_t$를 구한다. 계산 과정은 길지만 결과는 위의 깔끔한 공식이다.

3.3 뉴럴넷은 무엇을 학습하는가?

$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon)$을 $\tilde{\mu}_t$에 대입하면:

> **이 대입이 핵심 트릭이다**: 역과정의 평균 $\tilde{\mu}_t$는 x_0와 x_t 둘 다에 의존하는데, 실제 생성 시에는 x_0를 모른다 (그것을 만들어내는 것이 목표이니까!). 하지만 Forward 과정의 폐쇄형에서 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$이므로, x_0를 ϵ으로 대체할 수 있다. 즉, "x_0를 모르지만 ϵ을 알면 x_0를 복원할 수 있다"는 것이다.

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\epsilon\right)$$

대입 과정 상세:

$\tilde{\mu}_t$의 원래 공식에 $x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(x_t - \sqrt{1-\bar{\alpha}_t}\epsilon)$을 대입:

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t} \cdot \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$$

x_t 계수를 모으면:

$$= x_t\left[\frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\right] - \epsilon \cdot \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t\sqrt{1-\bar{\alpha}_t}}{(1-\bar{\alpha}_t)\sqrt{\bar{\alpha}_t}}$$

$\bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$을 이용하여 정리하면 (중간 대수 생략):

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right)$$

모델 $\epsilon_\theta(x_t, t)$가 추가된 노이즈 ϵ을 예측하면, 예측 평균은:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\epsilon_\theta(x_t, t)\right)$$

> **세 가지 예측 패러다임**: 뉴럴넷이 예측할 수 있는 대상은 세 가지이다:
>
> 1. **ϵ-prediction** (DDPM): 추가된 노이즈를 예측 → 가장 안정적 학습
> 2. **x_0-prediction**: 원본 이미지를 직접 예측 → 직관적이지만 학습이 어려움
> 3. **v-prediction** (Salimans & Ho, 2022): $v = \sqrt{\bar{\alpha}_t}\epsilon - \sqrt{1-\bar{\alpha}_t}x_0$ → 고해상도에서 더 안정적
>
> 세 가지는 수학적으로 등가이며 서로 변환 가능하다. DDPM 논문은 ϵ-prediction이 경험적으로 가장 좋다는 것을 보여주었다.

3.4 학습 목표 (Simple Loss)

> **ELBO에서 Simple Loss까지의 여정 — 왜 이렇게 단순해지는가?**
>
> DDPM의 학습 목표는 VAE와 마찬가지로 ELBO (Evidence Lower Bound)에서 시작한다. VAE에서 배운 것처럼, $\log p(x_0)$를 직접 최대화하기 어려우므로 하한(lower bound)을 최대화한다.
>
> **Step 1: ELBO 분해**
>
> $$\log p(x_0) \geq \mathbb{E}_q\left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right] = -\mathcal{L}_{\text{VLB}}$$
>
> **Step 2: VLB를 타임스텝별로 분해**
>
> $$\mathcal{L}_{\text{VLB}} = \underbrace{D_{KL}(q(x_T|x_0)\|p(x_T))}_{\mathcal{L}_T\ (\text{상수, 학습 불필요})} + \sum_{t=2}^{T}\underbrace{D_{KL}(q(x_{t-1}|x_t, x_0)\|p_\theta(x_{t-1}|x_t))}_{\mathcal{L}_{t-1}} + \underbrace{(-\log p_\theta(x_0|x_1))}_{\mathcal{L}_0}$$
>
> $\mathcal{L}_T$는 Forward 과정의 마지막 분포와 사전 분포의 KL이므로 θ에 무관하여 무시한다.
>
> **Step 3: 각 $\mathcal{L}_{t-1}$은 두 가우시안의 KL divergence**
>
> $q(x_{t-1}|x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$와 $p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta, \tilde{\beta}_t I)$의 KL divergence는 (분산이 같으므로):
>
> $$\mathcal{L}_{t-1} = \frac{1}{2\tilde{\beta}_t}\|\tilde{\mu}_t - \mu_\theta\|^2 + \text{const}$$
>
> **Step 4: μ를 ϵ으로 변환**
>
> $\tilde{\mu}_t$와 μ_θ에 ϵ 표현을 대입하면:
>
> $$\mathcal{L}_{t-1} = \frac{\beta_t^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}\|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

> **Step 5: Simple Loss (Ho et al.의 핵심 발견)**
>
> 앞의 가중치 $\frac{\beta_t^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}$는 t에 따라 달라지는데, Ho et al.은 이 가중치를 **그냥 1로 놓는 것**이 오히려 학습이 잘 된다는 것을 발견했다! 이론적 최적이 아닌데도 경험적으로 더 좋은 이유는: 가중치를 제거하면 큰 노이즈 레벨(큰 t)에서의 손실이 상대적으로 커져, 모델이 전체적 구조를 먼저 학습하고 세부 디테일을 나중에 학습하는 "교육과정(curriculum)" 효과가 생기기 때문이다.

$$\boxed{\mathcal{L}_{\text{simple}} = \mathbb{E}_{t\sim U[1,T],\, x_0,\, \epsilon\sim\mathcal{N}(0,I)}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]}$$

학습 알고리즘:

```
반복:
  1. 훈련 데이터에서 x₀ 샘플
  2. t ~ Uniform{1, ..., T} 랜덤 선택
  3. ε ~ N(0, I) 랜덤 노이즈 생성
  4. x_t = √ᾱ_t · x₀ + √(1-ᾱ_t) · ε  (한 번에 노이즈 이미지 생성)
  5. Loss = ‖ε - ε_θ(x_t, t)‖²  (실제 노이즈 vs 예측 노이즈)
  6. 역전파로 θ 업데이트
```

> **이 학습 알고리즘이 놀라운 이유**: 위 알고리즘은 결국 "노이즈가 섞인 이미지에서 노이즈를 예측하는 회귀(regression) 문제"이다. GAN처럼 두 네트워크를 적대적으로 학습시킬 필요도 없고, VAE처럼 ELBO의 두 항을 균형 맞출 필요도 없다. 단순한 MSE loss 하나로 학습이 완료된다. 이 놀라운 단순함이 DDPM의 실용적 성공의 핵심 비결이다.

3.5 생성 알고리즘

```
x_T ~ N(0, I)  (순수 노이즈에서 시작)
for t = T, T-1, ..., 1:
    z ~ N(0, I) if t > 1, else z = 0
    x_{t-1} = (1/√α_t)(x_t - β_t/√(1-ᾱ_t) · ε_θ(x_t, t)) + √β̃_t · z
return x₀
```

> **생성 알고리즘의 직관적 이해**: 각 스텝에서 하는 일을 풀어보면:
>
> 1. 현재 노이즈 이미지 x_t에서 뉴럴넷으로 "어떤 노이즈가 섞여 있는지" 예측 ($\epsilon_\theta(x_t, t)$)
> 2. 예측된 노이즈를 빼서 한 스텝 더 깨끗한 이미지 x_{t-1}을 복원
> 3. (마지막 스텝이 아니면) 약간의 새 노이즈 z를 추가하여 확률적 다양성 유지
>
> 새 노이즈 z를 추가하는 이유: 이것이 없으면 결정론적(deterministic)이 되어 같은 x_T에서 항상 같은 x_0가 나온다. z를 추가하면 매번 다른 결과가 나오며, 이것이 SDE의 확률적 성격이다. 나중에 배울 DDIM은 바로 이 z를 제거($\sigma = 0$)하여 결정론적 생성을 달성한다.
>
> **왜 1000 스텝이 필요한가?** 각 스텝에서 제거하는 노이즈 양이 β_t로 제한되기 때문이다. β_t가 작으므로 (0.0001~0.02), 한 번에 많은 노이즈를 제거할 수 없다. 이것은 SDE를 이산화(discretize)할 때 스텝 크기가 작아야 정확한 것과 같은 원리이다. Section 7(Flow Matching)과 Section 11(Consistency Models)은 이 문제를 해결하는 방법을 다룬다.

3.6 Score-Based Model과의 수학적 동치 관계

강의 5에서 배운 스코어와 DDPM의 노이즈 예측은 변환 관계에 있다:

$$\boxed{s_\theta(x_t, t) = \nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{\sqrt{1-\bar{\alpha}_t}} \cdot \frac{1}{\sqrt{1-\bar{\alpha}_t}} = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}}$$

단계별 유도:

$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1-\bar{\alpha}_t) I)$이므로:

$$\log q(x_t|x_0) = -\frac{1}{2(1-\bar{\alpha}_t)} \|x_t - \sqrt{\bar{\alpha}_t} x_0\|^2 + \text{const}$$

x_t에 대해 미분하면:

$$\nabla_{x_t} \log q(x_t|x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{1-\bar{\alpha}_t}$$

$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$이므로 $x_t - \sqrt{\bar{\alpha}_t} x_0 = \sqrt{1-\bar{\alpha}_t}\epsilon$을 대입:

$$= -\frac{\sqrt{1-\bar{\alpha}_t}\epsilon}{1-\bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}$$

따라서:

$$s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}$$

> **핵심**: 스코어를 학습하든 노이즈를 예측하든 수학적으로 같다. DDPM은 노이즈 예측이 학습이 더 안정적이라는 것을 보여주었다. 이유는: 스코어의 크기는 t에 따라 크게 변하는데 (t가 작으면 스코어가 매우 큼), 노이즈 ϵ은 항상 $\mathcal{N}(0, I)$에서 왔으므로 크기가 일정하여 학습이 안정적이다.

3.7 네트워크 구조: U-Net

DDPM은 **U-Net** 구조를 사용한다:

```
x_t, t
  ↓
[Time Embedding (sinusoidal → MLP)]
  ↓
[Encoder]  ──skip connection──→  [Decoder]
 ↓ conv, downsample               ↑ conv, upsample
 ↓ ResBlock + Attention           ↑ ResBlock + Attention
 ↓                                ↑
 └──────→ [Bottleneck] ──────────┘
  ↓
ε^(예측 노이즈, 입력과 동일 크기)
```

- **Time Embedding**: Transformer의 positional encoding과 동일한 sinusoidal 임베딩으로 타임스텝 t를 조건으로 주입
- **Skip Connection**: 인코더의 고해상도 정보를 디코더에 전달 → 세밀한 디테일 유지
- **Self-Attention**: 중간 해상도(16x16 등)에서 글로벌 관계 모델링

> **왜 U-Net인가?** DDPM의 입출력은 동일한 크기이다: 노이즈 이미지 x_t를 입력받아 같은 크기의 노이즈 ϵ을 출력한다. 이런 "동일 해상도 입출력" 태스크에 U-Net이 적합한 이유:
>
> 1. **인코더 경로**가 이미지를 점진적으로 다운샘플링하면서 "무엇이 있는지" (의미적 정보)를 파악
> 2. **디코더 경로**가 업샘플링하면서 "어디에 있는지" (공간 정보)를 복원
> 3. **Skip connection**이 인코더의 고해상도 특징을 디코더에 직접 전달하여 세밀한 디테일 보존
>
> **Time Embedding의 작동 방식**: 타임스텝 t는 정수 (1~1000)인데, 이를 Transformer의 positional encoding과 같은 sinusoidal 함수로 벡터화한 후, MLP를 통과시켜 각 ResBlock에 주입한다. 이렇게 하면 네트워크가 "현재 노이즈 레벨이 얼마인지"를 알 수 있고, 이에 맞는 디노이징 전략을 적용할 수 있다. t가 크면 (노이즈가 많으면) 전체적 구조를 복원하는 데 집중하고, t가 작으면 (거의 깨끗하면) 세밀한 디테일을 다듬는다.

📄 **U-Net 원본**: Ronneberger et al., "U-Net: Convolutional Networks for Biomedical Image Segmentation", MICCAI 2015 — 원래 의료 영상 세그멘테이션용으로 제안되었으나, DDPM에서 노이즈 예측 네트워크로 채택되었다.

---

4. Diffusion Transformer (DiT)

> MIT 6.S978: Kaiming He 직접 교수, 핵심 연구 주제

4.1 U-Net의 한계와 DiT의 등장

DDPM은 U-Net을 사용했지만, LLM에서 Transformer의 스케일링 법칙이 검증된 이후, "Diffusion에도 Transformer를 쓰면 어떨까?"라는 질문이 나왔다.

> **왜 U-Net에서 Transformer로 전환해야 했는가?** U-Net은 DDPM과 Stable Diffusion 1.x~2.x에서 훌륭하게 작동했지만, 근본적인 한계가 있었다:
>
> 1. **스케일링의 어려움**: U-Net은 인코더-디코더 구조에 Skip Connection이 복잡하게 얽혀 있어, 모델을 키울 때 어떤 부분을 확장해야 할지 불명확하다 (채널 수? 레이어 수? 해상도 단계 수?). 반면 Transformer는 "레이어를 쌓고 hidden dimension을 키우면 성능이 올라간다"는 명확한 스케일링 법칙이 GPT-3, PaLM 등에서 이미 검증되었다.
> 2. **인덕티브 바이어스(inductive bias)의 차이**: CNN 기반 U-Net은 지역적 패턴(local pattern)에 강하지만, 이미지의 장거리 의존성(long-range dependency)을 포착하려면 Self-Attention 레이어를 중간에 추가해야 한다. Transformer는 모든 레이어에서 글로벌 어텐션을 수행하므로, 이미지 전체의 일관성을 더 잘 유지한다.
> 3. **하드웨어 효율성**: 최신 GPU/TPU는 행렬 곱셈에 최적화되어 있어, Transformer의 어텐션 연산이 CNN의 불규칙한 메모리 접근보다 더 효율적으로 실행된다.
>
> DiT는 이 모든 문제를 "순수 Transformer 구조"로 해결하여, 디퓨전 모델에서도 LLM과 같은 스케일링 법칙이 작동함을 증명했다.

📄 **핵심 논문**: Peebles & Xie, "Scalable Diffusion Models with Transformers" (DiT), ICCV 2023

4.2 DiT의 구조

```
노이즈 잠재 표현 z_t (32x32x4)
    ↓
[Patchify: 2x2 패치 → 256개 토큰]
    ↓
[+Position Embedding]
    ↓
[DiT Block xN]
  ├─ LayerNorm (adaptive: γ, β를 t, class 조건으로 생성)
  ├─ Self-Attention
  ├─ LayerNorm (adaptive)
  └─ MLP (Feed-Forward)
    ↓
[Unpatchify]
    ↓
ε^또는 v 예측
```

> **Patchify가 왜 필요한가?** Transformer는 토큰(token)의 시퀀스를 입력으로 받는다. 이미지를 토큰으로 변환하려면 ViT(Vision Transformer)에서 사용한 Patchify 기법을 적용한다: 32x32 잠재 표현을 2x2 패치로 나누면 16x16 = 256개의 토큰이 된다. 각 토큰은 2x2x4 = 16차원의 벡터를 Linear layer로 hidden_dim(예: 1152)차원으로 사영한다. 패치 크기를 키우면 토큰 수가 줄어 연산이 빨라지지만, 공간 해상도 정보가 손실된다. DiT 논문은 패치 크기 2, 4, 8을 비교하여 2가 최적임을 보여주었다.

핵심 기법 — **adaLN-Zero** (Adaptive Layer Normalization):

$$\mathrm{adaLN}(h, y) = \gamma(y) \odot \mathrm{LayerNorm}(h) + \beta(y)$$

- y: 조건 벡터 (timestep t + class label의 결합)
- $\gamma(y), \beta(y)$: MLP로 조건에서 생성한 스케일/시프트 파라미터
- **Zero 초기화**: 학습 초기에 블록 출력을 0으로 시작 → 안정적 학습

> **왜 adaLN-Zero가 다른 조건 주입 방법보다 좋은가?** DiT 논문은 네 가지 조건 주입 방식을 비교했다:
>
> 1. **In-context conditioning**: 조건을 추가 토큰으로 시퀀스에 연결 → 가장 단순하지만 성능 최하
> 2. **Cross-attention**: 조건을 별도 시퀀스로 두고 Cross-Attention → 파라미터 많고 복잡
> 3. **adaLN** (Adaptive LayerNorm): 조건에서 γ, β를 동적 생성 → 파라미터 적고 효과적
> 4. **adaLN-Zero**: adaLN + 출력 게이트 α의 Zero 초기화 → **최고 성능**
>
> **Zero 초기화의 핵심 직관**: 학습 시작 시 모든 DiT 블록의 출력 게이트 α가 0이면, 전체 네트워크는 입력을 그대로 통과시키는 항등 함수(identity function)가 된다. 이것은 ResNet의 잔차 연결(residual connection)과 같은 효과로, "아무것도 하지 않는 상태"에서 점진적으로 유용한 변환을 학습한다. GPT 등 대형 Transformer에서도 유사한 초기화 기법이 학습 안정성에 핵심적이다.

4.3 왜 DiT가 중요한가?

	U-Net (DDPM/LDM)	DiT
구조	CNN + Attention 혼합	**순수 Transformer**

스케일링	구조 변경 필요	파라미터 수만 늘리면 됨
FID (ImageNet 256)	~3.6 (LDM)	**2.27** (DiT-XL/2)
후속 모델	Stable Diffusion 1,2	**Stable Diffusion 3, FLUX, Sora**

> **DiT의 스케일링 법칙이 검증된 방법**: DiT 논문은 DiT-S (33M), DiT-B (130M), DiT-L (458M), DiT-XL (675M) 네 가지 크기의 모델을 학습시키고, 모든 크기에서 일관되게 "모델이 크면 FID가 좋아진다"는 것을 보여주었다. 이것은 LLM에서의 스케일링 법칙(Kaplan et al., 2020)과 동일한 패턴이며, "더 큰 모델 = 더 좋은 생성"이라는 단순한 레시피가 이미지 생성에서도 통한다는 것을 의미한다.

> **현재 SOTA 이미지/비디오 생성 모델은 거의 모두 DiT 기반.**

4.4 PyTorch 구현: adaLN-Zero Block

⚠️ **참고**: 아래는 DiT의 핵심인 adaLN-Zero 블록의 구현이다. 전체 DiT 모델은 이 블록을 N번 쌓고, Patchify/Unpatchify 레이어를 추가한다.

```python
import torch
import torch.nn as nn

class AdaLNZeroDiTBlock(nn.Module):
    """DiT의 핵심 블록: adaLN-Zero (Adaptive Layer Normalization with Zero I

    일반 Transformer 블록과의 차이점:
    1. LayerNorm의 γ, β를 조건 벡터(timestep + class)에서 동적 생성
    2. 각 블록 출력에 스케일 팩터 α를 곱하되, 초기값을 0으로 설정
       → 학습 초기에 블록이 항등 함수처럼 동작하여 안정적 학습

    수식: adaLN(h, y) = γ(y) ⊙ LayerNorm(h) + β(y)
    """

    def __init__(self, hidden_dim=1152, num_heads=16, cond_dim=1152):
        super().__init__()
        self.hidden_dim = hidden_dim

        # --- Self-Attention ---
        self.norm1 = nn.LayerNorm(hidden_dim, elementwise_affine=False)
        self.attn = nn.MultiheadAttention(
            embed_dim=hidden_dim,
            num_heads=num_heads,
            batch_first=True
        )

        # --- Feed-Forward Network (MLP) ---
        self.norm2 = nn.LayerNorm(hidden_dim, elementwise_affine=False)
        self.mlp = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim * 4),
            nn.GELU(),                              # DiT는 GELU 활성화 사용
            nn.Linear(hidden_dim * 4, hidden_dim)
        )

        # --- adaLN 파라미터 생성 MLP ---
        # 조건 벡터 y → 6개의 파라미터: (γ1, β1, α1, γ2, β2, α2)
        # γ: 스케일, β: 시프트, α: 블록 출력 게이트
        self.adaLN_modulation = nn.Sequential(
            nn.SiLU(),                              # Swish 활성화
            nn.Linear(cond_dim, 6 * hidden_dim)     # 6개 파라미터 한 번에 생
        )

        # ★ Zero 초기화: 핵심 트릭!
        # 마지막 Linear 레이어의 가중치와 편향을 0으로 초기화
        # → 학습 시작 시 α1=0, α2=0이므로 블록 출력이 0
        # → 전체 네트워크가 항등 함수처럼 동작 → 안정적 학습 시작
        nn.init.zeros_(self.adaLN_modulation[-1].weight)
        nn.init.zeros_(self.adaLN_modulation[-1].bias)

    def forward(self, x, conditioning):
        """
        Args:
            x: 토큰 시퀀스 [B, N, D] (N=패치 수, D=hidden_dim)
            conditioning: 조건 벡터 [B, D] (timestep + class embedding)

        Returns:
```

```
        x: 업데이트된 토큰 시퀀스 [B, N, D]
    """
    # 조건에서 adaLN 파라미터 생성 (6개)
    modulation = self.adaLN_modulation(conditioning)  # [B, 6*D]
    gamma1, beta1, alpha1, gamma2, beta2, alpha2 = modulation.chunk(6,
    # 각각 [B, D] 크기

    # --- Self-Attention + adaLN ---
    # 1. LayerNorm (학습 가능한 γ,β 없음 → elementwise_affine=False)
    h = self.norm1(x)
    # 2. 조건부 스케일/시프트 적용: γ1 ⊙ LN(x) + β1
    h = gamma1.unsqueeze(1) * h + beta1.unsqueeze(1)
    # 3. Self-Attention
    h, _ = self.attn(h, h, h)
    # 4. 게이트(α1)로 출력 스케일 조절 (Zero Init → 초기에 0)
    x = x + alpha1.unsqueeze(1) * h

    # --- MLP + adaLN ---
    h = self.norm2(x)
    h = gamma2.unsqueeze(1) * h + beta2.unsqueeze(1)
    h = self.mlp(h)
    x = x + alpha2.unsqueeze(1) * h

    return x


# --- 사용 예시 ---
# B, N, D = 4, 256, 1152  # 배치, 패치 수 (32/2=16, 16*16=256), 은닉 차원
# block = AdaLNZeroDiTBlock(hidden_dim=D, num_heads=16, cond_dim=D)
#
# x = torch.randn(B, N, D)              # 패치 토큰
# t_emb = torch.randn(B, D)             # 타임스텝 임베딩
# class_emb = torch.randn(B, D)         # 클래스 라벨 임베딩
# conditioning = t_emb + class_emb      # 조건 벡터 결합
#
# output = block(x, conditioning)        # [B, N, D]
```

---

5. Latent Diffusion & Stable Diffusion 상세

> MIT 6.S978 Lecture 5~6, Stanford CS236과 MIT 6.S184에서도 핵심 주제

📄 **핵심 논문**: Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

5.1 픽셀 vs 잠재 공간 디퓨전

	Pixel Diffusion (DDPM)	**Latent Diffusion (LDM)**
디퓨전 공간	$256 \times 256 \times 3 = 196{,}608$차원	$32 \times 32 \times 4 = 4{,}096$차원
압축률	없음	**48배 압축**
학습 속도	느림	빠름

고해상도	거의 불가능	512, 1024 이상 가능

> **왜 잠재 공간에서 디퓨전을 수행하는가? — 핵심 통찰** 이미지의 대부분의 정보는 고주파(high-frequency) 디테일이다: 에지, 텍스처, 미세한 색상 변화 등. 이것들은 인간에게 중요하지만, 이미지의 "의미적 내용"(무엇이 어디에 있는지)은 저주파(low-frequency) 정보에 담겨 있다. LDM의 핵심 통찰은 이 두 가지를 분리하는 것이다:
>
> - **오토인코더(AutoEncoder)**: 고주파 디테일의 인코딩/디코딩을 담당. 이것은 한 번만 학습하면 재사용 가능.
> - **디퓨전 모델**: 잠재 공간에서 의미적 구조만 학습. 차원이 48배 줄어들었으므로 연산량이 극적으로 감소.
>
> 비유하자면, 고해상도 사진을 전송할 때 원본 그대로 보내는 것(Pixel Diffusion)과 JPEG으로 압축한 후 보내는 것(Latent Diffusion)의 차이이다. JPEG은 인간이 눈치채기 어려운 고주파 성분을 버리고 핵심 정보만 유지한다.

⚠️ **중요**: LDM의 핵심 통찰은 "고주파 디테일은 인코더/디코더에 맡기고, 디퓨전은 의미적 구조만 학습한다"는 것이다. 이 분리 덕분에 계산 비용이 대폭 감소한다.

5.2 LDM 파이프라인

```
                          학습 Phase 1: AutoEncoder
이미지 x → [Encoder E] → 잠재 표현 z → [Decoder D] → 복원 x̂
           (VQ-GAN 또는 KL-VAE)

                          학습 Phase 2: Diffusion in Latent Space
z₀ → [노이즈 추가] → z_t → [U-Net/DiT: ε_θ(z_t, t, c)] → ε̂→ Loss

                          생성 (Inference)
텍스트 → [CLIP/T5 Encoder] → 조건 c
                                    ↓
노이즈 z_T → [Denoise xT] → z₀ → [Decoder D] → 이미지
```

> **두 단계(Phase) 학습이 왜 중요한가?**
>
> **Phase 1 (AutoEncoder)**: 오토인코더는 이미지를 잠재 공간으로 압축하고 복원하는 역할이다. Stable Diffusion에서는 VQ-GAN 기반의 오토인코더를 사용하며, 이것은 강의 4에서 배운 VQ-GAN과 동일한 구조이다. 핵심은 이 오토인코더가 **디퓨전과 독립적으로** 학습된다는 것이다. 한 번 잘 학습된 오토인코더는 다양한 디퓨전 모델에서 재사용할 수 있다. SD 1.x, 2.x, SDXL은 모두 같은 오토인코더를 공유한다.
>
> **Phase 2 (Latent Diffusion)**: 오토인코더가 고정된 후, 잠재 공간에서 DDPM과 동일한 디퓨전 학습을 수행한다. 차이점은 입력이 픽셀 이미지 x가 아니라 잠재 표현 $z = E(x)$라는 것뿐이다. 수식은 완전히 동일하다: $\mathcal{L} = \|\epsilon - \epsilon_\theta(z_t, t, c)\|^2$.
>
> **왜 분리하면 좋은가?** 오토인코더 학습은 복원 품질에만 집중하고, 디퓨전 학습은 생성 품질에만 집중할 수 있다. 이 "관심의 분리(separation of concerns)"가 전체 시스템의 학습을 훨씬 효율적으로 만든다.

5.3 텍스트 조건 주입: Cross-Attention

U-Net/DiT에서 텍스트 조건을 주입하는 핵심 메커니즘:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

여기서:

- $Q = W_Q \cdot \phi_i(z_t)$ → 이미지 잠재 표현에서 Query
- $K = W_K \cdot \tau_\theta(y)$ → **텍스트 인코더 출력**에서 Key
- $V = W_V \cdot \tau_\theta(y)$ → **텍스트 인코더 출력**에서 Value

> **Cross-Attention의 직관적 이해**: Self-Attention에서는 Q, K, V가 모두 같은 입력에서 나온다. Cross-Attention에서는 Q는 "이미지 측"에서, K와 V는 "텍스트 측"에서 나온다. 이것의 의미를 풀어보면:
>
> - **Q (Query)**: 이미지의 각 공간 위치가 "나는 어떤 정보가 필요한가?"를 질문
> - **K (Key)**: 텍스트의 각 토큰이 "나는 이런 정보를 가지고 있다"라고 응답
> - **V (Value)**: K가 매칭되면 실제로 전달되는 정보
>
> 예를 들어, "빨간 자동차가 도로 위에 있다"라는 텍스트에서, 이미지의 중앙 위치가 "자동차" 토큰에 높은 어텐션을 주면, 자동차의 시각적 정보가 그 위치로 전달된다. "빨간"이라는 토큰은 색상 정보를 제공하여 해당 영역이 빨간색으로 생성된다.
>
> **$\sqrt{d}$로 나누는 이유**: Q와 K의 내적 값이 차원 d에 비례하여 커지면, softmax가 거의 one-hot 분포가 되어 학습이 어려워진다. $\sqrt{d}$로 나누면 내적의 분산이 1로 정규화되어 softmax가 적절한 범위에서 작동한다.

> **강의 2의 CogView/Crayon에서 배운 Cross-Attention과 동일한 메커니즘.** DALL-E의 decoder-only 단일 시퀀스 방식보다 이 Cross-Attention 방식이 더 효과적이어서 표준이 되었다.

5.4 Stable Diffusion 버전별 진화

버전	년도	Backbone	텍스트 인코더	핵심 변화
SD 1.x	2022.08	U-Net	CLIP ViT-L/14	LDM 원본
SD 2.x	2022.11	U-Net	OpenCLIP ViT-H	v-prediction
SDXL	2023.07	U-Net (확대)	CLIP + OpenCLIP 듀얼	두 텍스트 인코더
SD 3	2024.02	**MM-DiT**	CLIP + T5-XXL 트리플	**Flow Matching** + DiT
FLUX	2024.08	DiT 변형	T5-XXL	Black Forest Labs

> **SD 1.x → SD 3로의 진화에서 알 수 있는 트렌드**:
>
> 1. **Backbone**: U-Net → DiT (Section 4에서 배운 이유로)
> 2. **텍스트 인코더**: 단일 CLIP → CLIP + T5 조합 (Section 9에서 배울 이유로)
> 3. **학습 프레임워크**: DDPM → Flow Matching (Section 7에서 배울 이유로)
> 4. **일관된 방향**: 각 구성 요소가 독립적으로 개선되면서 전체 시스템이 발전. 이것이 모듈화된 LDM 파이프라인의 강점이다.

📄 **SD 3 논문**: Esser et al., "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis", ICML 2024

6. Classifier-Free Guidance (CFG)

> 사실상 모든 조건부 생성 모델의 핵심 기법. MIT 6.S184 Section 5에서 별도 다룸.

6.1 동기

조건부 생성 $p(x \mid c)$에서, 조건 c를 더 강하게 따르게 하려면?

> **왜 Guidance가 필요한가?** 디퓨전 모델을 조건부로 학습하면 $p(x|c)$를 모델링한다. 하지만 실제로 샘플링하면, 생성된 이미지가 조건을 "느슨하게" 따르는 경우가 많다. 예를 들어, "빨간 자동차"라고 했는데 주황색에 가까운 자동차가 나오거나, 배경에 불필요한 물체가 등장한다. 이유는 $p(x|c)$가 높은 영역이 넓어서, 조건과 완전히 일치하지 않는 샘플도 확률적으로 나올 수 있기 때문이다. Guidance는 이 분포를 "날카롭게" 만들어 조건에 더 충실한 샘플을 생성한다.

베이즈 정리로:

$$\nabla_x \log p(x \mid c) = \nabla_x \log p(x) + \nabla_x \log p(c \mid x)$$

이 분해의 의미: 조건부 스코어 = 무조건부 스코어 + 분류기 그래디언트. 즉, 조건 c를 따르려면 "기본 생성 방향"에 "조건 c의 확률이 높아지는 방향"을 더하면 된다.

Classifier Guidance (Dhariwal & Nichol, 2021): 별도의 분류기 $p(c \mid x)$의 기울기를 추가. 하지만 별도 분류기 학습이 필요하다는 단점.

> **Classifier Guidance의 구체적 문제점**:
>
> 1. 노이즈가 있는 이미지 x_t에 대해 작동하는 분류기를 학습해야 한다 (일반 분류기는 깨끗한 이미지에서만 학습)
> 2. 분류기의 기울기가 디퓨전 모델의 학습 분포와 맞지 않으면 아티팩트 발생
> 3. 분류기 학습에 별도의 레이블 데이터와 학습 시간이 필요
> 4. 텍스트 조건처럼 복잡한 조건에는 분류기를 설계하기 어려움

📄 **Classifier Guidance 논문**: Dhariwal & Nichol, "Diffusion Models Beat GANs on Image Synthesis", NeurIPS 2021

6.2 Classifier-Free Guidance

📄 **핵심 논문**: Ho & Salimans, "Classifier-Free Diffusion Guidance", NeurIPS Workshop 2022

별도 분류기 없이, **동일한 모델**로 조건부/무조건부를 모두 학습:

학습 시: 일정 확률(예: 10%)로 조건 c를 빈 조건 $\varnothing$로 대체하여 학습

$$\text{학습 시 입력} = \begin{cases} \epsilon_\theta(x_t, t, c) & \text{확률 } 0.9 \\ \epsilon_\theta(x_t, t, \varnothing) & \text{확률 } 0.1 \end{cases}$$

> **왜 조건을 랜덤으로 드롭하는가?** 이것은 Dropout과 유사한 정규화(regularization) 기법이다. 조건 c를 10% 확률로 빈 조건 $\varnothing$으로 대체하면, 모델은 동시에 두 가지를 학습하게 된다: (1) 조건이 주어졌을 때의 생성 $\epsilon_\theta(x_t, t, c)$, (2) 조건 없이의 생성 $\epsilon_\theta(x_t, t, \varnothing)$. 하나의 모델

> 이 두 역할을 모두 수행할 수 있는 이유는, $\varnothing$도 하나의 "조건"으로 취급하기 때문이다 (예: 빈 텍스트 임베딩, 또는 학습된 null 토큰).

생성 시: 조건부와 무조건부 예측의 **가중 외삽**:

$$\boxed{\tilde{\epsilon}_\theta(x_t, t, c) = \epsilon_\theta(x_t, t, \varnothing) + w \cdot (\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing))}$$

이 수식의 단계별 해석:

1. $\epsilon_\theta(x_t, t, \varnothing)$: "조건 없이 일반적으로 어떤 이미지를 생성할까?" (무조건부 예측)
2. $\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)$: "조건 c가 있으면 없을 때와 비교해 **방향이 얼마나 달라지는가?**" (조건의 효과)
3. $w \cdot (\ldots)$: 이 "조건의 효과"를 w배로 **증폭**
4. 최종: 무조건부 예측 + w배 증폭된 조건 효과

수식을 다시 정리하면:

$$\tilde{\epsilon} = (1 - w) \cdot \epsilon_\theta(x_t, t, \varnothing) + w \cdot \epsilon_\theta(x_t, t, c)$$

- $w = 1$: 표준 조건부 생성 (조건부 예측을 그대로 사용)
- $w > 1$ (보통 3~15): 조건에 **더 충실**한 생성 (외삽/과장)
- $w = 0$: 무조건부 생성 (조건 무시)

> **왜 $w > 1$이 "외삽"인가?** $w = 1$이면 정확히 조건부 예측 $\epsilon_\theta(x_t, t, c)$이다. $w > 1$이면 조건부 예측을 "넘어서" 조건의 방향으로 더 나아간다. 이것은 보간(interpolation, $0 \leq w \leq 1$)이 아니라 외삽(extrapolation, $w > 1$)이다. 기하학적으로, 무조건부 점에서 조건부 점을 향한 벡터를 따라 조건부 점을 지나쳐 더 멀리 가는 것이다.
>
> **w의 효과를 직관적으로**: w가 크면 "텍스트 조건에 과하게 충실한" 이미지가 생성된다. 이미지가 더 선명하고 텍스트에 일치하지만, 동시에 다양성이 줄고 과포화(over-saturation)가 발생할 수 있다. 이것은 정확도-다양성의 트레이드오프(fidelity-diversity trade-off)이며, w가 이 트레이드오프를 조절하는 "다이얼"이다.

⚠️ **주의**: w 값이 너무 크면 이미지가 과포화(over-saturated)되거나 아티팩트가 생긴다. 실전에서는 $w = 7.5$ (SD 1.x 기본값) ~ $w = 3.5$ (FLUX 기본값) 정도가 적절하다.

6.3 기하학적 해석

```
                        w=1 (표준)
                       ↗
무조건부 예측 ──→ 조건부 예측 ──→ w=7.5 (강한 가이던스)
 ε(x_t, ∅)        ε(x_t, c)         ε̃(x_t, c)
```

조건과 무조건의 "방향 차이"를 w배로 확대하여 외삽한다.

> **스코어 공간에서의 기하학적 해석**: 스코어 함수 공간에서 생각하면, $\nabla_x \log p(x)$는 데이터 분포의 "밀도가 높아지는 방향"을 가리킨다. CFG는 이 방향 벡터를 조작한다:
>
> $$\tilde{s}(x, c) = s(x, \varnothing) + w \cdot (s(x, c) - s(x, \varnothing))$$

> 이것은 스코어 공간에서 "무조건부 밀도 피크"에서 "조건부 밀도 피크"를 향하는 벡터를 w배로 늘리는 것이다. w가 크면 조건부 밀도 피크를 지나쳐 "더 극단적으로 조건에 부합하는" 방향으로 이동한다.
>
> **이것이 implicit classifier와 같은 이유**: CFG 수식을 정리하면:
>
> $$\tilde{s}(x, c) = s(x, \varnothing) + w \cdot (s(x, c) - s(x, \varnothing))$$
>
> $s(x, c) - s(x, \varnothing) = \nabla_x \log p(x|c) - \nabla_x \log p(x) = \nabla_x \log \frac{p(x|c)}{p(x)} = \nabla_x \log p(c|x) + \text{const}$
>
> 즉, $s(x, c) - s(x, \varnothing)$는 사실상 **암시적 분류기(implicit classifier)** $p(c|x)$의 그래디언트이다! 별도 분류기를 학습하지 않았는데도, 조건부/무조건부 모델의 차이가 자동으로 분류기 역할을 한다.

6.4 Negative Prompt

실전에서 Stable Diffusion의 Negative Prompt가 작동하는 원리:

$$\tilde{\epsilon} = \epsilon_\theta(x_t, t, c_{\text{neg}}) + w \cdot (\epsilon_\theta(x_t, t, c_{\text{pos}}) - \epsilon_\theta(x_t, t, c_{\text{neg}}))$$

c_{neg}: "blurry, low quality" 같은 부정 프롬프트. 빈 조건 $\varnothing$ 대신 피하고 싶은 조건을 넣는다.

> **Negative Prompt의 직관**: 원래 CFG에서 $\varnothing$ (빈 조건)은 "아무 조건 없는 일반 이미지"를 의미한다. Negative Prompt는 이를 "피하고 싶은 이미지"로 대체한다. 수식의 의미를 풀어보면:
>
> "c_{neg}로 향하는 방향에서 벗어나, c_{pos}로 향하는 방향으로 w배 더 강하게 이동하라"
>
> 예를 들어, c_{pos} = "선명한 고양이 사진", c_{neg} = "흐릿한 저화질"이면, 모델은 "흐릿함 → 선명함" 방향으로 강하게 이동한다. 이것은 c_{neg}가 $\varnothing$일 때보다 더 효과적인데, "흐릿함"이라는 구체적 반대 방향을 제공하기 때문이다.
>
> **실전 팁**: Negative Prompt에 "worst quality, low quality, blurry, deformed"를 넣는 것이 일반적이며, 이것만으로도 생성 품질이 크게 향상된다.

---

7. Flow Matching

> MIT 6.S184 핵심 주제 (2025, 2026 모두), CMU 10-799 별도 과목까지 개설

📄 **핵심 논문**: Lipman et al., "Flow Matching for Generative Modeling", ICLR 2023

7.1 Diffusion의 문제점

DDPM/Score-Based Model은 **1000 스텝**의 반복 샘플링이 필요하다. 왜?

- SDE(확률 미분방정식) 기반 → 경로가 구불구불
- 직선으로 못 가고 우회 → 많은 스텝이 필요

> **왜 DDPM의 경로가 구불구불한가? — 수학적 이유** DDPM의 역과정은 SDE를 시뮬레이션한다: $dx = [\ldots]dt + g(t)d\bar{W}$. 여기서 $d\bar{W}$는 브라운 운동(Brownian motion)의 미소 증분인

> 데, 브라운 운동은 본질적으로 "무작위 지그재그"이다. 각 스텝에서 결정론적 방향(스코어를 따라 데이터 쪽으로)과 확률적 노이즈($d\bar{W}$)가 결합되므로, 전체 경로는 "대략적으로는 올바른 방향이지만 미시적으로는 무작위"인 구불구불한 궤적이 된다.
>
> 이것은 물리학에서 꽃가루가 물 위에서 움직이는 브라운 운동과 같다: 전체적으로는 한 방향으로 이동하지만, 실제 경로는 무수히 많은 지그재그로 이루어져 있다. 이산 시뮬레이션에서 스텝 크기가 크면 이 지그재그를 제대로 포착하지 못해 오차가 커지므로, 작은 스텝(= 많은 반복)이 필요하다.

7.2 핵심 아이디어: 직선 경로

Flow Matching은 노이즈 → 데이터를 **최대한 직선**으로 연결한다.

Diffusion (SDE):

```
x₀ ~~~ 구불구불한 확률적 경로 ~~~> x_T
```

Flow Matching (ODE):

```
x₀ ──────── 직선 경로 ────────> x₁
```

> **왜 직선 경로가 더 좋은가?** 직선 경로의 장점은 두 가지이다:
>
> 1. **적은 스텝**: 직선은 가장 짧은 경로이므로, Euler method 같은 간단한 수치 적분기로도 적은 스텝(10~50)으로 정확하게 시뮬레이션할 수 있다. 구불구불한 경로는 곡률이 큰 부분에서 스텝을 작게 해야 하므로 더 많은 스텝이 필요하다.
> 2. **결정론적**: ODE는 확률적 노이즈가 없으므로, 같은 시작점에서 항상 같은 결과를 얻는다. 이것은 재현성(reproducibility)과 보간(interpolation)에 유리하다.
>
> 비유: 서울에서 부산까지 가는 두 가지 방법을 생각하자. SDE는 "대략 남쪽으로 가되, 매 교차로에서 동전을 던져 좌우를 결정하는" 방식이다 (결국 도착하지만 경로가 길다).
> ODE/Flow Matching은 "GPS가 알려주는 최단 경로를 따라가는" 방식이다.

7.3 수학적 프레임워크

ODE (상미분방정식)로 정의:

$$\frac{dx_t}{dt} = v_\theta(x_t, t), \quad t \in [0, 1]$$

- $t = 0$: 순수 노이즈 $x_0 \sim \mathcal{N}(0, I)$
- $t = 1$: 깨끗한 데이터 $x_1 \sim p_{\text{data}}$
- v_θ: **속도장**(velocity field). 뉴럴넷이 학습.

> **ODE vs SDE의 핵심 차이**:
>
> | | SDE (DDPM) | ODE (Flow Matching) |
> |---|---|---|
> | 수식 | $dx = f(x,t)dt + g(t)dW$ | $dx/dt = v(x,t)$ |

노이즈 항	$g(t)dW$ (있음)	없음
경로	확률적 (매번 다름)	결정론적 (항상 같음)
샘플링	MCMC 특성	수치 적분

속도장 $v_\theta(x_t, t)$의 의미: 시각 t에서 위치 x_t에 있는 입자가 "어느 방향으로 얼마나 빠르게" 움직여야 하는지를 알려주는 벡터장이다. 물리학에서 유체의 속도장과 정확히 같은 개념이다 (그래서 이 분야를 "Continuous Normalizing Flow"라고도 부른다). 뉴럴넷 v_θ는 이 속도장을 학습하여, 노이즈 분포에서 출발한 입자가 시간 $t = 1$에서 데이터 분포에 도달하도록 "안내"한다.

7.4 조건부 Flow Matching (Conditional FM)

왜 "조건부" Flow Matching이 필요한가? 이상적으로는 주변(marginal) 속도장 $v_t(x) = \mathbb{E}[v_t(x|x_1)|x_t = x]$를 직접 학습하고 싶지만, 이 기댓값을 계산하려면 모든 데이터 포인트에 대한 적분이 필요하여 계산 불가능하다. Lipman et al.의 핵심 기여는 "조건부 속도장을 학습해도 결국 주변 속도장과 같은 효과를 낸다"는 것을 증명한 것이다. 즉, 각 데이터 포인트 x_1에 대한 간단한 조건부 경로를 독립적으로 학습하면 충분하다.

각 데이터 포인트 x_1과 노이즈 x_0 쌍에 대해 조건부 경로를 정의:

$$\psi_t(x_0 \mid x_1) = (1-t)x_0 + tx_1 \quad (\text{선형 보간})$$

이 경로의 속도장:

$$u_t(x \mid x_1) = x_1 - x_0$$

이 속도장이 상수인 이유: $\psi_t = (1-t)x_0 + tx_1$를 t에 대해 미분하면 $d\psi_t/dt = -x_0 + x_1 = x_1 - x_0$이다. 즉, 조건부 경로가 직선이면 속도장은 시간에 무관한 상수 벡터이다. 이것은 "노이즈 x_0에서 데이터 x_1까지의 방향과 거리"를 나타내며, 시간에 관계없이 일정하다.

학습 목표 (Conditional Flow Matching Loss):

$$\boxed{\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t\sim U[0,1],\, x_0\sim\mathcal{N}(0,I),\, x_1\sim p_{\text{data}}}\left[\|v_\theta(\psi_t, t) - (x_1 - x_0)\|^2\right]}$$

DDPM의 $\|\epsilon - \epsilon_\theta\|^2$와 형식이 거의 동일! 단, 노이즈가 아니라 **속도(방향)**를 예측한다.

DDPM Loss와의 정밀 비교:

	DDPM	**Flow Matching**
입력	$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$	$\psi_t = (1-t)x_0 + tx_1$
예측 대상	ϵ (추가된 노이즈)	$x_1 - x_0$ (속도/방향)
시간 범위	$t \in \{1, \ldots, T\}$ (이산)	$t \in [0, 1]$ (연속)
노이즈 스케줄	β_t (설계 필요)	불필요 (선형 보간)

Flow Matching의 실용적 장점: 노이즈 스케줄 β_t를 설계할 필요가 없다. DDPM에서는 linear, cosine, scaled linear 등 다양한 스케줄을 실험해야 했지만, Flow Matching은 단순 선형 보간으로 충분하다.

7.5 Flow Matching vs DDPM 비교

	DDPM	Flow Matching
수학적 기반	SDE (확률 미분방정식)	**ODE** (상미분방정식)
경로	구불구불 (확률적)	**직선에 가까움**
학습 목표	ϵ (노이즈) 예측	v (속도) 예측
샘플링 스텝	1000 (DDPM), 50 (DDIM)	**10~50** (Euler method)
대표 모델	SD 1.x, 2.x	**SD 3, FLUX, Movie Gen**
이론	Score Matching	**Continuous Normalizing Flow**

7.6 Optimal Transport (OT) 경로

Flow Matching의 강점: Optimal Transport 경로를 사용할 수 있다.

$$\psi_t^{\mathrm{OT}} = (1 - (1 - \sigma_{\min})t)\, x_0 + t\, x_1$$

> **Optimal Transport란 무엇인가?** OT(최적 수송)는 "한 분포를 다른 분포로 변환하는 데 필요한 최소 비용 경로"를 찾는 수학적 이론이다. 직관적으로, 모래더미를 다른 모양으로 옮기는 데 가장 적은 노동력이 드는 방법을 찾는 것이다.
>
> 기본 Flow Matching에서는 각 데이터 포인트 x_1이 랜덤한 노이즈 x_0와 쌍을 이루는데, OT-CFM에서는 "가장 가까운" 노이즈-데이터 쌍을 매칭한다. 이렇게 하면 경로가 서로 교차하지 않아 학습이 더 안정적이고, 적은 스텝으로 더 좋은 결과를 얻는다.

OT 경로는 노이즈에서 데이터까지의 **가장 효율적인 이동 경로**를 제공한다. 이 때문에 더 적은 스텝으로 좋은 결과를 얻는다.

📄 **OT-CFM 논문**: Tong et al., "Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport", ICML 2024

7.7 Diffusion과 Flow Matching의 통합

최근 연구(diffusionflow.github.io)에 따르면:

> **Gaussian Flow Matching = Diffusion Model**. 수학적으로 동일하며, 다른 네트워크 출력(속도 vs 노이즈)과 다른 샘플링 스케줄을 사용할 뿐이다.

$$v_t = \frac{d\psi_t}{dt} \quad \longleftrightarrow \quad \epsilon_t = -\sigma_t \cdot s_\theta(x_t, t) \quad \longleftrightarrow \quad s_\theta = \nabla_{x_t} \log p_t(x_t)$$

세 가지 예측 대상은 서로 변환 가능하다:

예측 대상	모델	변환
ϵ (노이즈)	DDPM	$s = -\epsilon/\sigma_t$
s (스코어)	Score-Based	$\epsilon = -\sigma_t \cdot s$
v (속도)	Flow Matching	$v = \alpha_t' x_t/\alpha_t + \sigma_t' \sigma_t s$

> **이 통합이 왜 중요한가?** 처음에는 DDPM, Score-Based Model, Flow Matching이 완전히 다른 모델처럼 보였다. 하지만 이 통합 관점에서 보면, 이들은 **같은 수학적 대상의 다른 표현**일 뿐이다. 마치 벡터를 직교좌표 (x, y)로 표현하든 극좌표 (r, θ)로 표현하든 같은 벡터인 것처럼, 노이즈/스코어/속도는 같은 "디노이징 방향"의 다른 파라미터화(parameterization)이다.
>
> 실용적 의미: 한 프레임워크에서 학습한 모델을 다른 프레임워크의 샘플러로 샘플링할 수 있다. 예를 들어, ϵ-prediction으로 학습한 모델의 출력을 v로 변환하여 Flow Matching 스타일의 Euler 샘플러로 빠르게 생성할 수 있다.

---

8. SDE/ODE 통합 프레임워크

> MIT 6.S184 전체 과정의 이론적 뼈대, Stanford CS236 Lecture 16

8.1 Yang Song의 통합 (ICLR 2021)

Score-Based Model과 DDPM을 **SDE**로 통합:

📄 **핵심 논문**: Song et al., "Score-Based Generative Modeling through Stochastic Differential Equations", ICLR 2021 (Outstanding Paper Award)

> **왜 통합이 필요했는가?** 2020~2021년, 생성 모델 분야에는 두 진영이 있었다: (1) Song & Ermon의 Score-Based Model (NCSN, SMLD), (2) Ho et al.의 DDPM. 두 모델은 외견상 매우 다르게 보였지만, Yang Song은 두 모델이 모두 SDE의 특수한 경우임을 보여주었다. 이 통합은 단순한 이론적 호기심이 아니라, 실용적으로도 큰 의미가 있었다: (1) 두 모델의 장점을 결합할 수 있게 되었고, (2) SDE 이론의 풍부한 도구들(수치 해석, 최적 제어 등)을 생성 모델에 적용할 수 있게 되었으며, (3) 확률 흐름 ODE를 발견하여 빠른 샘플링(DDIM)의 이론적 기반을 제공했다.

Forward SDE:

$$dx = f(x, t)\, dt + g(t)\, dW$$

- $f(x, t)$: 드리프트 계수 (결정론적 성분) — 데이터에 체계적으로 가해지는 변화
- $g(t)$: 확산 계수 (노이즈 강도) — 각 시점에서 추가되는 노이즈의 크기
- W: 브라운 운동 (Brownian Motion) — 연속 시간의 무작위 노이즈

> **SDE 수식의 직관적 이해**: SDE를 이산 시간으로 근사하면 $x_{t+\Delta t} \approx x_t + f(x_t, t)\Delta t + g(t)\sqrt{\Delta t}\, z$ ($z \sim \mathcal{N}(0, I)$)이다. 이것은 "현재 위치에서 드리프트 방향으로 조금 이동하고, 랜덤 노이즈를 조금 추가한다"는 의미이다. $\Delta t \to 0$의 극한이 연속 시간 SDE이다.
>
> **브라운 운동 W의 성질**: dW는 $\mathcal{N}(0, dt)$를 따르는 무작위 증분이다. 핵심 성질은 $\mathbb{E}[dW] = 0$ (평균 0), $\mathbb{E}[(dW)^2] = dt$ (분산이 시간 증분에 비례)이다. 이 때문에 $\sqrt{\Delta t}$ 항이 등장하며, 이것이 SDE를 ODE와 본질적으로 다르게 만든다.

Reverse SDE (Anderson, 1982):

$$dx = \left[f(x, t) - g(t)^2 \nabla_x \log p_t(x)\right] dt + g(t)\, d\bar{W}$$

스코어 $\nabla_x \log p_t(x)$를 알면 역과정을 시뮬레이션할 수 있다.

> **역 SDE가 왜 이런 형태인가?** Anderson(1982)은 Forward SDE $dx = fdt + gdW$의 시간을 뒤집으면, 역과정이 위의 형태가 됨을 증명했다. 핵심은 드리프트에 추가되는 $-g(t)^2\nabla_x \log p_t(x)$ 항이다. 이 항은 "현재 시점의 데이터 분포가 밀도가 높은 방향"을 가리킨다. 즉, Forward에서 노이즈가 추가되어 분포가 퍼진 것을 Reverse에서 스코어를 따라 다시 모아주는 것이다.

8.2 확률 흐름 ODE (Probability Flow ODE)

같은 주변 분포 $p_t(x)$를 가지는 결정론적 ODE가 존재한다:

$$\frac{dx}{dt} = f(x,t) - \frac{1}{2}g(t)^2\nabla_x \log p_t(x)$$

> **왜 "같은 주변 분포"인가? — 이것이 핵심이다** SDE와 ODE는 완전히 다른 경로를 따른다 (SDE는 확률적, ODE는 결정론적). 하지만 놀랍게도, 시각 t에서 입자들의 **분포** $p_t(x)$는 동일하다. 비유하자면: SDE는 "각 사람이 취한 채로 걷는" 것이고, ODE는 "각 사람이 술 취한 군중의 밀도를 보고 합리적으로 이동하는" 것이다. 개인의 경로는 다르지만, 전체 군중의 분포는 동일하다.
>
> **수학적 이유**: Fokker-Planck 방정식에 의해, SDE의 주변 분포는 특정 편미분방정식을 만족한다. 위 ODE의 주변 분포도 같은 편미분방정식을 만족하도록 설계되었다. 노이즈 항 $g(t)dW$가 사라지는 대신, 드리프트의 스코어 항이 $g^2\nabla \log p_t$에서 $\frac{1}{2}g^2\nabla \log p_t$로 절반이 된다.

- SDE: 확률적 샘플링 (매번 다른 경로) → 다양성 높음
- ODE: 결정론적 샘플링 → 재현 가능, 빠른 샘플링 (DDIM)

8.3 구체적 인스턴스

Forward SDE	**모델**	$f(x,t)$	$g(t)$
Variance Preserving	DDPM	$-\frac{1}{2}\beta(t)x$	$\sqrt{\beta(t)}$
Variance Exploding	NCSN/SMLD	0	$\sqrt{\frac{d[\sigma^2(t)]}{dt}}$
Sub-VP	—	$-\frac{1}{2}\beta(t)x$	$\sqrt{\beta(t)(1-e^{-2\int_0^t \beta(s)ds})}$

> **VP vs VE의 핵심 차이**:
>
> - **Variance Preserving (VP, DDPM)**: 드리프트 $f = -\frac{1}{2}\beta(t)x$가 원점으로 끌어당기면서 동시에 노이즈를 추가한다. 분산이 항상 1 근처로 유지되어 "분산이 보존"된다. 학습이 안정적이라는 장점이 있다.
> - **Variance Exploding (VE, NCSN)**: 드리프트가 0이고 노이즈만 추가한다. 시간이 지나면 분산이 무한히 커진다 ("분산이 폭발"). Song & Ermon의 원래 Score Matching 접근이 이 범주에 속한다.
>
> DDPM이 VP-SDE에 해당한다는 것은 Ho et al.의 이산 시간 Forward를 연속 시간으로 확장하면 자연스럽게 나온다.

8.4 DDIM은 ODE 솔버

DDIM (Denoising Diffusion Implicit Models, Song et al. 2020)은:

- DDPM의 확률 흐름 ODE를 이산화한 것
- $\sigma = 0$으로 설정하면 **결정론적** 생성 (같은 x_T → 같은 x_0)
- 50 스텝만으로 DDPM 1000 스텝에 근접하는 품질

> **DDIM이 ODE 솔버인 이유 — 상세 설명** DDIM의 업데이트 규칙을 살펴보면:
>
> $$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \epsilon_\theta(x_t, t) + \sigma_t z$$
>
> $\sigma_t = 0$으로 설정하면 노이즈 항 z가 사라지고, 업데이트가 완전히 결정론적이 된다. 이것은 정확히 확률 흐름 ODE를 Euler method로 이산화한 것과 같다. $\sigma_t > 0$이면 SDE 성격이 추가되며, $\sigma_t = \sqrt{\tilde{\beta}_t}$일 때 원래 DDPM과 동일하다.
>
> **왜 50 스텝으로 충분한가?** ODE는 SDE보다 경로가 매끄러우므로, 큰 스텝으로도 정확한 시뮬레이션이 가능하다. DDIM은 1000개 타임스텝 중 50개만 등간격으로 선택하여 "건너뛰기" 하는데, ODE의 매끄러운 경로 덕분에 이 근사가 잘 작동한다.

📄 **DDIM 논문**: Song et al., "Denoising Diffusion Implicit Models", ICLR 2021

---

9. CLIP & 멀티모달 조건부 생성

> MIT 6.S978 세미나 논문, Kaiming He 연구 그룹의 관련 연구

9.1 CLIP (Contrastive Language-Image Pre-training)

📄 **핵심 논문**: Radford et al., "Learning Transferable Visual Models From Natural Language Supervision", ICML 2021

> **왜 CLIP이 생성 모델에 핵심적인가?** Text-to-Image 생성에서 가장 근본적인 문제는 "텍스트와 이미지를 어떻게 연결하는가?"이다. 텍스트 "빨간 자동차"와 빨간 자동차 이미지는 완전히 다른 데이터 형태(문자열 vs 픽셀 배열)인데, 이 둘 사이의 의미적 관계를 모델이 이해해야 한다. CLIP은 4억 개의 텍스트-이미지 쌍으로 학습하여, 텍스트와 이미지를 **같은 벡터 공간**에 매핑하는 방법을 학습했다. 이 공유 공간에서 "빨간 자동차"라는 텍스트 벡터와 빨간 자동차 이미지 벡터는 가까이 위치한다. 이 공유 임베딩 공간이 Stable Diffusion의 텍스트 조건 주입(Cross-Attention)을 가능하게 하는 핵심 기반이다.

```
이미지 x_i → [Image Encoder (ViT)] → 이미지 임베딩 z_i^{img}
                                                    ↕ 코사인 유사도
텍스트 t_i → [Text Encoder (Transformer)] → 텍스트 임베딩 z_i^{txt}
```

Contrastive Learning 목표:

배치 내 N개의 (이미지, 텍스트) 쌍에서:

- 맞는 쌍: 유사도를 높임
- 틀린 쌍: 유사도를 낮춤

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[\log\frac{\exp(\text{sim}(z_i^{img}, z_i^{txt})/\tau)}{\sum_{j=1}^{N}\exp(\text{sim}(z_i^{img}, z_j^{txt})/\tau)}\right]$$

> **Contrastive Loss의 상세 이해**:
>
> 이 수식은 **InfoNCE Loss** (Oord et al., 2018)의 한 형태이다. 직관적으로 해석하면:
>
> - **분자** $\exp(\text{sim}(z_i^{img}, z_i^{txt})/\tau)$: 올바른 (이미지, 텍스트) 쌍의 유사도. 이것이 커야 한다.
> - **분모** $\sum_j \exp(\text{sim}(z_i^{img}, z_j^{txt})/\tau)$: 이미지 i와 배치 내 **모든** 텍스트의 유사도 합. 올바른 쌍과 틀린 쌍을 모두 포함.
> - **전체**: $-\log(\text{분자}/\text{분모})$를 최소화하면, 올바른 쌍의 유사도는 높이고 틀린 쌍의 유사도는 낮아진다.
>
> **온도 파라미터** τ: τ가 작으면 softmax가 더 날카로워져 "가장 유사한 쌍 하나"에 집중하고, τ가 크면 더 부드러워져 여러 쌍에 분산된다. CLIP은 τ도 학습 가능한 파라미터로 설정한다.
>
> **배치 크기의 중요성**: 배치 내 N개의 텍스트 중 $N-1$개가 "네거티브 샘플"이다. N이 클수록 더 어려운 네거티브를 만나므로 학습이 효과적이다. CLIP은 **배치 크기 32,768**을 사용하여 매우 강력한 contrastive learning을 달성했다.
>
> **대칭 손실**: 위 수식은 "이미지 → 텍스트" 방향이지만, CLIP은 "텍스트 → 이미지" 방향도 동시에 학습한다. 최종 손실은 두 방향의 평균이다.

- 4억 개의 텍스트-이미지 쌍으로 학습
- Zero-shot 이미지 분류: ImageNet에서 별도 학습 없이 76.2% 정확도

9.2 CLIP이 생성 모델에서 사용되는 방식

모델	CLIP 사용 방식
DALL-E 2	CLIP 이미지 임베딩을 예측하는 Prior + Diffusion Decoder
Stable Diffusion 1.x	CLIP ViT-L/14 텍스트 인코더 → Cross-Attention
SDXL	CLIP ViT-L + OpenCLIP ViT-bigG → 듀얼 인코더
Imagen	T5-XXL 텍스트 인코더 (CLIP 대신)
SD 3	CLIP ViT-L + CLIP ViT-bigG + T5-XXL → 트리플 인코더

> **Stable Diffusion에서 CLIP이 어떻게 사용되는가 — 구체적 흐름**:
>
> 1. 사용자가 텍스트 프롬프트 입력 (예: "a cat sitting on a red car")
> 2. CLIP Text Encoder가 텍스트를 토큰화하고, 각 토큰을 768차원(ViT-L) 또는 1280차원(ViT-bigG) 벡터로 인코딩
> 3. 최대 77개 토큰의 시퀀스가 생성됨 (shape: [77, 768])
> 4. 이 시퀀스가 U-Net/DiT의 Cross-Attention 레이어에서 Key와 Value로 사용
> 5. 디퓨전 과정 중 매 스텝에서 이미지 특징(Query)이 텍스트 특징(Key/Value)과 어텐션을 수행
>
> **핵심**: CLIP Text Encoder는 생성 과정 중 고정(frozen)된다. 디퓨전 모델만 학습하고, CLIP의 가중치는 변경하지 않는다. 이것이 가능한 이유는 CLIP이 이미 충분히 풍부한 텍스트 표현

> 을 학습했기 때문이다.

📄 **DALL-E 2 논문**: Ramesh et al., "Hierarchical Text-Conditional Image Generation with CLIP Latents", 2022

📄 **Imagen 논문**: Saharia et al., "Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding", NeurIPS 2022

9.3 왜 T5가 CLIP을 보완하는가?

- CLIP: 이미지-텍스트 정합에 최적화 → 시각적 개념에 강함
- T5: 순수 언어 모델로 학습 → 복잡한 문장 이해, 속성 바인딩에 강함

> **CLIP의 한계와 T5의 보완 — 구체적 예시**: CLIP은 "이미지와 텍스트의 전체적 의미가 맞는지"를 학습했다. 따라서 "고양이"와 고양이 이미지의 매칭에는 강하다. 하지만 다음과 같은 복잡한 문장에서는 약하다:
>
> - **속성 바인딩 실패**: "빨간 모자를 쓴 파란 개" → CLIP은 "빨간", "모자", "파란", "개"를 각각 인식하지만, "빨간이 모자에, 파란이 개에 연결된다"는 관계를 제대로 인코딩하지 못할 수 있다. 결과: 파란 모자를 쓴 빨간 개가 생성될 수 있다.
> - **공간 관계 실패**: "고양이 왼쪽에 개" → CLIP은 공간 관계를 잘 인코딩하지 못한다.
> - **수량 실패**: "세 마리의 고양이" → CLIP은 수량 개념이 약하다.
>
> T5는 순수 언어 모델로서 문장의 **구문 구조**를 깊이 이해한다. "빨간 모자를 쓴 파란 개"에서 형용사-명사 관계를 정확히 파악한다. SD 3에서 CLIP + T5를 결합함으로써, 시각적 개념(CLIP)과 언어적 구조(T5) 두 가지를 모두 활용할 수 있게 되었다.

SD 3에서 두 종류를 모두 사용하여 각각의 강점을 결합한다.

---

10. Visual Autoregressive Models (VAR, MAR)

> MIT 6.S978: Kaiming He의 핵심 연구, NeurIPS 2024 Best Paper (VAR)

10.1 기존 Visual AR 모델의 한계

DALL-E 식 접근 (강의 5):

```
이미지 → [VQ-VAE] → 래스터 순서 토큰 (좌→우, 위→아래)
t₁ → t₂ → t₃ → ... → t₁₀₂₄
```

문제: 이미지는 2D 공간 구조를 가지는데, 1D로 펼치면 공간 정보가 손실된다.

> **왜 래스터 순서가 문제인가? — 구체적 예시** 32x32 토큰 맵을 좌→우, 위→아래로 펼치면, 공간적으로 바로 아래에 있는 토큰은 시퀀스에서 32칸 떨어져 있다. Transformer의 Self-Attention은 이론적으로 모든 위치를 참조할 수 있지만, 자기회귀 모델에서는 **이전 토큰만** 참조 가능하다. 따라서 1행의 마지막 토큰을 생성할 때 2행의 첫 토큰 정보는 사용할 수 없다. 이것은 이미지의 수직적 일관성(예: 건물의 수직선, 사람의 몸체)을 유지하기 어렵게 만든다.

> 또한, 래스터 순서 AR은 **생성 속도가 느리다**: 1024개 토큰을 하나씩 순차 생성해야 하므로 1024번의 forward pass가 필요하다.

10.2 VAR (Visual Autoregressive Modeling)

📄 **핵심 논문**: Tian et al., "Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction", NeurIPS 2024 **Best Paper**

핵심 아이디어: 토큰 단위가 아니라 **해상도(스케일) 단위**로 자기회귀 생성.

```
[1x1 토큰 맵] → [2x2 토큰 맵] → [4x4] → [8x8] → [16x16] → [32x32]
 (전체 구조)      (대략적 형태)    ...     ...      ...      (세밀한 디테일)
```

> **왜 Next-Scale Prediction이 Next-Token Prediction보다 좋은가?** GPT 스타일의 "다음 토큰 예측"은 텍스트에서 훌륭하게 작동한다. 텍스트는 본질적으로 1D 시퀀스이므로 좌→우 순서가 자연스럽다. 하지만 이미지는 2D이며, "다음 픽셀"이라는 자연스러운 순서가 없다.
>
> VAR의 "다음 스케일 예측"은 인간의 인지 과정과 더 일치한다: 화가가 그림을 그릴 때, 먼저 대략적인 구도(composition)를 잡고, 그 위에 점점 세밀한 디테일을 추가한다. VAR도 마찬가지로:
>
> 1. 1x1 (1개 토큰): 이미지의 전체 색조/분위기
> 2. 2x2 (4개 토큰): 대략적 구도 (하늘/땅 구분 등)
> 3. 4x4 → 8x8 → ...: 점점 세밀한 구조
> 4. 32x32 (1024개 토큰): 최종 디테일
>
> 각 스케일에서 **한 번에** 해당 스케일의 모든 토큰을 병렬 생성하므로, 스케일 수(~10단계)만큼의 forward pass로 충분하다. 래스터 순서의 1024번 대비 극적으로 빠르다.

각 스케일 s_k에서 이전 스케일들이 모두 조건으로 주어진다:

$$p(x) = p(s_1) \cdot p(s_2 \mid s_1) \cdot p(s_3 \mid s_1, s_2) \cdots p(s_K \mid s_1, \ldots, s_{K-1})$$

장점:

- 2D 공간 구조를 자연스럽게 보존
- Coarse-to-fine 생성 (사람의 그림 그리는 과정과 유사)
- **LLM의 스케일링 법칙이 처음으로 이미지 생성에서도 관찰됨**
- FID 1.73 (ImageNet 256x256) — DiT를 능가

10.3 MAR (Masked Autoregressive Model)

📄 **핵심 논문**: Li et al. (Kaiming He), "Autoregressive Image Generation without Vector Quantization", NeurIPS 2024

핵심 아이디어: VQ(벡터 양자화) 없이, **연속 공간**에서 자기회귀 생성.

기존 방식: 이산 토큰 → Cross-Entropy Loss MAR: **연속 토큰** → 각 토큰의 분포를 **Diffusion Loss**로 모델링

$$\mathcal{L}_{\text{diffusion}}(x_i) = \mathbb{E}_{\epsilon,t}\left[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_i + \sqrt{1-\bar{\alpha}_t}\epsilon, t, c_i)\|^2\right]$$

여기서 c_i는 이전 토큰들로부터 계산된 조건 벡터.

> **왜 VQ 없이 AR을 하려는가?** VQ-VAE/VQ-GAN의 벡터 양자화는 본질적으로 **정보 손실**을 수반한다: 연속 벡터를 유한한 코드북의 가장 가까운 벡터로 대체하므로, 미세한 디테일(색상의 미묘한 차이, 텍스처의 연속적 변화)이 손실된다. 또한 코드북 크기가 유한하므로 표현력에 한계가 있다.
>
> MAR의 핵심 혁신은 각 토큰의 **확률 분포를 Diffusion으로 모델링**하는 것이다. 기존 AR은 이산 토큰에 대해 Cross-Entropy ($-\log p(t_i|t_{<i})$)를 사용했지만, MAR은 연속 토큰 x_i에 대해 Diffusion Loss를 사용한다. 이것은 AR(순차 생성)과 Diffusion(연속 분포 모델링)을 결합한 것으로, 두 패러다임의 장점을 모두 취한다.
>
> **Kaiming He의 MAE와의 연결**: MAR은 마스크된 토큰을 예측하는 Masked Autoencoder(MAE) 구조를 사용하되, 마스크된 토큰의 수를 점진적으로 줄여가며 자기회귀적으로 생성한다.

> **의미**: DALL-E의 "이산 토큰 → Cross-Entropy" 패러다임에서 벗어나, 연속 공간에서 AR + Diffusion을 결합. 강의 5에서 배운 DALL-E의 한계를 극복한 최신 접근.

---

11. Consistency Models & 가속 샘플링

> MIT 6.S184 (2026): Lecture 4에서 별도 다룸

> **왜 1-스텝 생성이 중요한가?** Diffusion 모델의 가장 큰 실용적 약점은 **생성 속도**이다. DDPM은 1000스텝, DDIM은 50스텝, Flow Matching은 10~50스텝이 필요하다. 각 스텝마다 전체 신경망을 한 번씩 순전파(forward pass)해야 하므로, 1000스텝 DDPM으로 512x512 이미지 하나를 생성하는 데 고성능 GPU에서도 수십 초가 걸린다.
>
> 이것은 **실시간 응용**에서 치명적이다:
>
> - **대화형 이미지 편집**: 사용자가 프롬프트를 바꿀 때마다 수십 초 대기
> - **비디오 생성**: 초당 30프레임 × 수백 프레임 = 수만 번의 디노이징
> - **로봇 제어**: Diffusion Policy에서 행동을 실시간으로 생성해야 함 (수 ms 이내)
> - **모바일/엣지 디바이스**: 서버 없이 로컬에서 생성해야 하는 경우
>
> GAN은 **1스텝**으로 생성할 수 있었지만 학습이 불안정했다. Consistency Model은 **Diffusion의 안정적 학습 + GAN의 빠른 생성**을 결합하려는 시도이다. 이것이 성공하면 Diffusion 모델의 마지막 약점이 해결되는 셈이다.

11.1 문제: 디퓨전의 느린 생성

DDPM: 1000 스텝, DDIM: 50 스텝, Flow Matching: 10~50 스텝.

"1 스텝으로 생성할 수 없을까?"

왜 디퓨전은 본질적으로 느린가? — 반복적 디노이징의 수학적 이유:

디퓨전 모델의 생성 과정은 SDE/ODE의 **이산 근사**(numerical discretization)이다. 연속 시간 ODE:

$$\frac{dx}{dt} = v_\theta(x_t, t)$$

를 실제로 풀려면 시간을 N개의 구간으로 나누고, 각 구간에서 Euler 방법 등으로 근사한다:

$$x_{t-\Delta t} = x_t + v_\theta(x_t, t) \cdot (-\Delta t)$$

여기서 N**이 작으면**(큰 스텝 크기) 근사 오차가 축적되어 이미지 품질이 급격히 떨어진다. 이것은 ODE 솔버의 근본적 한계이다. 직관적으로, 노이즈에서 이미지까지의 경로가 **곡선**이므로 큰 스텝으로 직선 근사하면 경로에서 이탈한다.

> **핵심 질문**: 곡선 경로를 따라가는 대신, 경로의 **시작점을 직접 예측**할 수는 없을까? 이것이 Consistency Model의 아이디어이다.

11.2 Consistency Models

> Song et al. (OpenAI), "Consistency Models", ICML 2023

핵심 아이디어 — 자기 일관성(Self-Consistency) 성질:

ODE 궤적(trajectory)은 결정론적이다. 즉, ODE 궤적 위의 **어떤 점에서 출발하든** 같은 $t = 0$ 지점(x_0, 즉 깨끗한 이미지)에 도달한다.

> **왜 ODE 궤적의 결정론적 성질이 핵심인가?** SDE(확률적 미분방정식)와 달리, ODE(상미분방정식)는 **결정론적**이다. 같은 초기조건에서 출발하면 항상 같은 경로를 따른다. 이것은 ODE 궤적 위의 모든 점 (x_t, t)가 **동일한** x_0**에 대응**한다는 뜻이다.
>
> ```
> ODE 궤적 시각화:
>
> x_T (노이즈) ──→ x_{t₃} ──→ x_{t₂} ──→ x_{t₁} ──→ x_0 (이미지)
>  │               │            │            │            │
>  └───────────────┴────────────┴────────────┴────────────┘
>          이 모든 점은 같은 x_0에 대응 (자기 일관성)
> ```
>
> 만약 함수 f가 이 성질을 학습한다면, x_t**에서 바로** x_0**를 예측**할 수 있다. 더 이상 중간 스텝을 밟을 필요가 없다!

ODE 궤적 위의 모든 점이 같은 x_0로 매핑되는 **일관성 함수(consistency function)** f_θ:

$$f_\theta(x_t, t) = f_\theta(x_{t'}, t') = x_0, \quad \forall t, t' \text{ on same trajectory}$$

경계 조건(Boundary Condition):

$$f_\theta(x_0, 0) = x_0$$

이 조건은 필수적이다: $t = 0$에서는 이미 깨끗한 이미지이므로, 항등 함수여야 한다. 실제 구현에서는 이 경계 조건을 **아키텍처에 내장**한다:

$$f_\theta(x, t) = \begin{cases} x & \text{if } t = 0 \\ c_{\text{skip}}(t) \cdot x + c_{\text{out}}(t) \cdot F_\theta(x, t) & \text{if } t > 0 \end{cases}$$

여기서 $c_{\text{skip}}(0) = 1$, $c_{\text{out}}(0) = 0$으로 설정하여 경계 조건을 자동으로 만족시킨다.

자기 일관성 손실(Self-Consistency Loss):

$$\mathcal{L} = \mathbb{E}\left[\|f_\theta(x_{t_{n+1}}, t_{n+1}) - f_{\theta^-}(x_{t_n}, t_n)\|^2\right]$$

> **이 손실 함수의 의미를 단계별로 이해하자**:
>
> **Step 1 — 인접 타임스텝 쌍 선택**: ODE 궤적 위에서 인접한 두 시점 t_{n+1}과 t_n을 선택한다. 이 두 점은 같은 궤적 위에 있으므로 **같은 x_0에 대응**해야 한다.
>
> **Step 2 — x_{t_n} 계산**: $x_{t_{n+1}}$에서 출발하여 ODE를 한 스텝 풀어서 x_{t_n}을 얻는다:
>
> $$x_{t_n} = x_{t_{n+1}} + (t_n - t_{n+1}) \cdot v_\theta(x_{t_{n+1}}, t_{n+1})$$
>
> 여기서 v_θ는 사전학습된 디퓨전 모델의 속도장/노이즈 예측기이다.
>
> **Step 3 — 일관성 강제**: $f_\theta(x_{t_{n+1}}, t_{n+1})$과 $f_{\theta^-}(x_{t_n}, t_n)$이 같아지도록 손실을 최소화한다. 두 입력이 같은 궤적 위에 있으므로, 일관성 함수의 출력도 같아야 한다.
>
> **Step 4 — EMA 타겟(θ^-)의 역할**: θ^-는 θ의 **지수 이동 평균**(Exponential Moving Average)이다:
>
> $$\theta^- \leftarrow \mu \cdot \theta^- + (1-\mu) \cdot \theta, \quad \mu \approx 0.999$$
>
> 왜 EMA가 필요한가? 만약 양쪽 모두 θ를 사용하면, 네트워크가 **모든 입력에 대해 같은 상수를 출력**하는 trivial solution으로 붕괴(collapse)할 수 있다. 한쪽을 EMA로 고정하면 이 붕괴를 방지한다. 이것은 Self-Supervised Learning에서 BYOL, MoCo 등이 사용하는 것과 동일한 기법이다.

- θ^-: EMA (Exponential Moving Average) 파라미터
- 인접 타임스텝의 출력이 일치하도록 학습

두 가지 학습 방식:

방식	설명	장단점
Consistency Distillation (CD)	사전학습된 디퓨전 모델의 ODE 궤적을 활용하여 x_{t_n}을 계산	기존 디퓨전 모델 필요, 더 안정적
Consistency Training (CT)	디퓨전 모델 없이, noising process로 직접 학습	독립적 학습 가능, 더 어려움

> **Consistency Distillation vs Training의 차이를 직관적으로 이해하면**:
>
> - **CD**: 이미 잘 학습된 디퓨전 모델(교사)이 ODE 궤적을 알려준다. 학생(Consistency Model)은 이 궤적 위의 점들이 같은 곳으로 매핑되도록 배운다. 교사가 있으므로 학습이 안정적이다.
> - **CT**: 교사 없이, 노이징 과정 $q(x_t|x_0)$으로 만든 (x_0, x_t) 쌍만으로 학습한다. 더 어렵지만, 사전학습된 디퓨전 모델이 불필요하므로 완전히 독립적이다.

결과: **1~4 스텝**으로 고품질 생성 가능.

다중 스텝 정제(Multi-Step Refinement)의 직관:

Consistency Model은 1스텝으로 x_0를 예측할 수 있지만, 예측이 완벽하지 않을 수 있다. 이때 **다중 스텝 정제**를 통해 품질을 올릴 수 있다:

```
1-스텝: x_T → f_θ(x_T, T) = x̂_0          (거친 예측)

2-스텝: x_T → f_θ(x_T, T) = x̂_0
        → 다시 노이즈 추가: x̂_{t₁} = √ᾱ_{t₁} x̂_0 + √(1-ᾱ_{t₁}) ε
        → f_θ(x̂_{t₁}, t₁) = x̂_0'        (정제된 예측)

일반적으로 2~4 스텝이면 DDPM 1000 스텝에 근접한 품질
```

> 이 과정의 직관: 1스텝 예측은 "대략적인 윤곽"을 잡고, 다시 약간의 노이즈를 추가한 후 디노이징하면 "세부 디테일"이 추가된다. 이것은 화가가 먼저 스케치를 그리고 점점 디테일을 추가하는 과정과 유사하다. 핵심은 **스텝 수를 사용자가 선택할 수 있다**는 것이다: 속도가 중요하면 1스텝, 품질이 중요하면 4스텝.

11.3 기타 가속 기법

기법	방법	스텝 수	핵심 아이디어
DDIM	확률 흐름 ODE의 이산화	50	SDE → ODE로 전환하여 결정론적 샘플링
DPM-Solver	고차 ODE 솔버	10~20	Euler 대신 2~3차 솔버로 정확도 향상
Progressive Distillation	교사 모델 → 학생 모델 (스텝 절반)	4~8	2스텝을 1스텝으로 증류, 반복 적용
Consistency Models	일관성 함수 학습	**1~4**	ODE 궤적의 자기 일관성 성질 활용
Consistency Distillation	사전학습 디퓨전 → Consistency 증류	**1~2**	기존 모델의 궤적을 교사로 활용

> **가속 기법들의 관계를 이해하면**:
>
> 이 기법들은 **ODE를 더 빠르게 푸는 방법**이라는 공통점이 있다:
>
> - **DDIM, DPM-Solver**: ODE **솔버를 개선** (더 정확한 수치 적분 → 적은 스텝으로 같은 정확도)
> - **Progressive Distillation**: **지식 증류**로 여러 스텝을 하나로 압축 (학생이 교사의 2스텝을 1스텝으로 모방)
> - **Consistency Models**: ODE를 풀지 않고, **궤적의 끝점을 직접 예측** (패러다임 전환)
>
> 최근에는 이들을 결합하는 연구도 활발하다: 예를 들어 **Latent Consistency Model (LCM)**은 Stable Diffusion의 잠재 공간에서 Consistency Distillation을 적용하여, SD를 4스텝으로 가속하면서 품질을 유지한다.

---

12. 비디오 생성 & Physical AI

> MIT 6.S978 세미나, MIT 6.S184 (2026) Lecture 3

> **왜 비디오 생성은 이미지 생성보다 본질적으로 어려운가?** 비디오는 이미지에 **시간 축(temporal axis)**이 추가된 것이지만, 단순히 차원이 하나 늘어난 것 이상의 도전이 있다:
>
> 1. **데이터 크기 폭발**: 256×256 이미지는 ~200KB이지만, 30fps × 10초 × 256×256 비디오는 ~60MB. 잠재 공간에서도 차원이 수십~수백 배 증가한다.
> 2. **시간적 일관성(Temporal Consistency)**: 인접 프레임이 부드럽게 연결되어야 한다. 이미지 모델을 프레임별로 독립 적용하면 **깜빡임(flickering)**이 발생한다. 이것은 각 프레임의 노이즈가 독립이므로 디노이징 결과도 프레임마다 다르기 때문이다.
> 3. **물리적 일관성**: 객체가 물리 법칙을 따라 움직여야 한다 (중력, 관성, 충돌 등). 이미지에서는 한 순간의 정적 장면만 그리면 되지만, 비디오에서는 **시간에 따른 역학(dynamics)**이 자연스러워야 한다.
> 4. **장기 의존성**: 비디오의 처음과 끝이 내러티브적으로 연결되어야 한다 (예: 공이 던져지면 나중에 떨어져야 함). 이것은 수백 프레임에 걸친 장거리 의존성을 요구한다.

12.1 이미지 → 비디오 확장

비디오 = 이미지 + **시간 축**. 핵심 구조적 변화:

```
이미지 DiT:   [B, C, H, W] → Spatial Attention
비디오 DiT:   [B, C, T, H, W] → Spatial Attention + Temporal Attention
```

> **Spatial + Temporal Attention의 분리를 직관적으로 이해하면**:
>
> 전체 3D 볼륨 $[T, H, W]$에 대해 Full Attention을 적용하면 계산량이 $O((T \cdot H \cdot W)^2)$으로 폭발한다. 256×256 이미지 30프레임이면 토큰 수가 ~2백만이 되어 현실적으로 불가능하다.
>
> 해결책: **Factorized Attention** — 공간과 시간을 분리하여 처리:
>
> ```
> 1단계: Spatial Attention (각 프레임 내부)
>  - 각 프레임을 독립적으로 처리
>  - 프레임 내 객체들의 공간적 관계 학습
>  - 복잡도: O(T × (H·W)²) — T개의 독립적인 이미지 어텐션
>
> 2단계: Temporal Attention (프레임 간 연결)
>  - 같은 공간 위치의 토큰들을 시간 축으로 어텐션
>  - 프레임 간 움직임, 변화 학습
>  - 복잡도: O(H·W × T²) — 각 위치에서 T 프레임 간 어텐션
>
> 총 복잡도: O(T·(H·W)² + H·W·T²) ≪ O((T·H·W)²)
> ```
>
> 이것은 이미지 생성 모델을 **최소한의 수정으로** 비디오로 확장할 수 있게 한다. 사전학습된 이미지 모델의 Spatial Attention 가중치를 유지하고, Temporal Attention만 새로 학습하는 것이 일반적이다.

시간적 일관성: 인접 프레임이 부드럽게 연결되어야 함.

12.2 주요 비디오 생성 모델

모델	기관	년도	핵심 구조
Video Diffusion Models	Google	2022	3D U-Net + Temporal Attention
Sora	OpenAI	2024.02	DiT + 시공간 패치
Movie Gen	Meta	2024.10	Flow Matching + DiT
VEO-3	Google	2025	멀티모달 + 대화 생성

Sora의 핵심 아이디어:

- **Spacetime Patches**: 비디오를 3D 패치로 분할 ($t \times h \times w$)
- 다양한 해상도와 길이를 동일 모델로 처리
- 텍스트 조건 + Flow Matching 기반 생성

> **Sora의 Spacetime Patches가 왜 혁신적인가?**
>
> 기존 비디오 모델들은 이미지를 2D 패치로 나누고 Temporal Attention을 추가하는 방식이었다. Sora는 **비디오를 처음부터 3D 패치로 분할**한다:
>
> ```
> 기존 방식:
>  비디오 [T, H, W] → 각 프레임을 2D 패치로 분할 → [T, N_h × N_w, D]
>  → Spatial Att. + Temporal Att. (분리)
>
> Sora 방식:
>  비디오 [T, H, W] → 3D 패치로 분할 [N_t × N_h × N_w, D]
>  → Full Spacetime Att. (통합)
>
>  각 패치는 (예: 4프레임 × 16×16 픽셀) 시공간의 한 "덩어리"
> ```
>
> 이 방식의 장점:
>
> 1. **가변 해상도/길이**: 패치 수만 조절하면 어떤 크기의 비디오도 처리 가능. 720p 60초도, 4K 5초도 같은 모델로 생성.
> 2. **시공간 통합 학습**: 공간과 시간을 분리하지 않으므로, 복잡한 3D 움직임(회전, 줌, 카메라 이동)을 더 자연스럽게 학습.
> 3. **DiT의 스케일링 법칙 계승**: Transformer의 스케일링 법칙이 비디오 생성에서도 작동함을 보여줌.

12.3 Diffusion Policy (로봇 행동 생성)

> Chi et al., "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion", RSS 2023

강의에서 "Physical AI에 필수"라고 한 영역:

```
관측 o_t (카메라 이미지 + 로봇 상태)
    ↓
[조건 인코더]
    ↓ (조건)
노이즈 a_T → [Denoise ×K] → 액션 시퀀스 a₀ = (a_t, a_{t+1}, ..., a_{t+H})
    ↓
로봇 실행
```

- 로봇의 행동(관절 각도, 이동)을 **확률 분포**로 모델링
- 동일 상황에서 여러 가능한 행동 → 다중 모드(multimodal) 분포 표현 가능
- 기존 행동 복제(Behavior Cloning)의 "평균 행동 문제" 해결

> **왜 로봇 행동에 Diffusion이 필요한가? — 다중 모드 분포 문제**:
>
> 로봇이 테이블 위의 컵을 잡으려 할 때, **왼쪽에서 잡는 것**과 **오른쪽에서 잡는 것** 모두 유효한 행동이다. 이 두 행동은 관절 각도 공간에서 완전히 다른 곳에 위치한다.
>
> ```
> 행동 분포 시각화 (1D 단순화):
>
> 확률
> ↑
> |    ┌─┐              ┌─┐
> |    │ │              │ │
> |    │ │              │ │
> |    │ │              │ │
> └────┴─┴──────────────┴─┴──→ 행동 (관절 각도)
>  왼쪽잡기          오른쪽잡기
>
> 기존 BC: 두 모드의 평균 → 컵을 정면에서 밀어버림 (위험!)
> Diffusion Policy: 두 모드 중 하나를 자연스럽게 샘플링
> ```
>
> 기존 **Behavior Cloning(BC)**은 L_2 손실로 학습하므로, 다중 모드 분포의 **평균**을 예측한다. 왼쪽 잡기와 오른쪽 잡기의 평균은 **컵을 정면에서 밀어버리는 행동**이 되어 위험하다!
>
> Diffusion은 본질적으로 **다중 모드 분포를 정확히 모델링**할 수 있다 (Section 3에서 배운 것처럼, Diffusion은 임의의 복잡한 분포를 학습할 수 있다). 따라서 로봇 행동 생성에 이상적이다.
>
> 추가적으로, Diffusion Policy는 **행동 시퀀스** $[a_t, a_{t+1}, ..., a_{t+H}]$ 전체를 한 번에 생성한다. 이것은 단일 행동을 예측하는 것보다 시간적으로 부드러운 궤적을 만든다.

12.4 World Models

환경을 시뮬레이션하는 비디오 예측 모델:

$$\hat{o}_{t+1} = f_\theta(o_t, a_t) \quad (\text{관측} + \text{행동} \rightarrow \text{다음 관측 예측})$$

- 물리 법칙을 학습하여 시뮬레이터 역할
- 로봇이 실제 행동 전에 "상상"으로 계획을 세울 수 있음
- NVIDIA의 Cosmos, Google의 Genie 등

> **왜 World Model이 중요한가? — "상상"으로 계획하기**:

> 인간은 행동하기 전에 **결과를 상상**한다. "이 컵을 세게 밀면 떨어지겠지"라는 내적 시뮬레이션이 있기에 조심스럽게 행동한다. World Model은 AI에게 이 능력을 부여한다.
>
> ```
> World Model의 활용 흐름:
>
> 현재 관측 o_t + 후보 행동 a_t
>  ↓
> World Model: ô_{t+1} = f_θ(o_t, a_t)    ← "이렇게 하면 이렇게 될 것"
>  ↓
> 여러 후보 행동에 대해 시뮬레이션
>  ↓
> 가장 좋은 결과를 주는 행동 선택
>  ↓
> 실제 행동 실행
> ```
>
> **비디오 생성 모델과의 연결**: World Model은 본질적으로 **조건부 비디오 생성**이다. 현재 프레임(관측)과 행동을 조건으로, 미래 프레임(다음 관측)을 생성한다. 따라서 Section 12.1~12.2에서 배운 비디오 생성 기술이 직접 활용된다.
>
> 이것이 **Physical AI**의 핵심이다: 디퓨전 모델이 단순히 예쁜 이미지를 만드는 것을 넘어, **물리 세계를 이해하고 행동을 계획**하는 도구가 된다.

> **의미**: Diffusion 모델이 이미지/비디오 생성을 넘어, 로봇 제어와 물리 시뮬레이션으로 확장되고 있다. 이것은 생성 모델이 "콘텐츠 생성 도구"에서 "세상을 이해하는 모델"로 진화하고 있음을 보여준다.

---

13. 종합 로드맵: 추천 학습 순서

> **왜 학습 순서가 중요한가?** 생성 모델은 서로 깊이 연결되어 있다. DDPM을 이해하지 않으면 LDM(Stable Diffusion)을 이해할 수 없고, Score Function을 모르면 Flow Matching이 왜 필요한지 알 수 없다. 또한 VAE를 모르면 잠재 공간(latent space)의 개념이 불분명해지고, GAN의 한계를 모르면 Diffusion의 혁신이 와닿지 않는다.
>
> 아래 로드맵은 **선수 지식이 자연스럽게 쌓이는 순서**로 구성되어 있다. 각 Phase를 건너뛰면 이후 내용의 이해가 표면적으로 머무를 수 있으므로, 가능한 순서대로 학습하기를 권장한다.

13.1 커리큘럼 매핑

아래 표는 각 주제가 어떤 강의에서 다뤄지는지를 보여준다:

주제	이 강의	CS236	6.S978	6.S184	CMU 10-423
VAE / VQ-VAE	L1,2	Wk3-4	L2	--	--
GAN / StyleGAN	L3,4	Wk5-6	L4	--	--
Autoregressive	L2,5	Wk2	L3	--	--
Normalizing Flow	L2	Wk4-5	--	--	--

EBM	--	Wk6-8	--	--	--
평가 지표	--	Wk8	pset	--	--
Score-Based	L5 개념	Wk7	--	L2	--
DDPM	수식 불충분	Wk10	**L5**	L1-2	--
DiT	--	--	**L6**	--	--
Latent Diffusion	--	--	**L5-6**	L3	--
CFG	--	--	pset	**L5**	--
Flow Matching	--	--	--	**L1-2**	--
SDE/ODE 통합	--	Wk10	--	**L1-2**	--
CLIP	--	--	세미나	--	--
VAR / MAR	--	--	**세미나**	--	--
Consistency Models	--	--	--	**L4**	--
비디오 생성	--	--	세미나	L3	--

> **이 표를 읽는 방법**: 각 강의는 서로 다른 관점에서 생성 모델을 다룬다.
>
> - **Stanford CS236 (Ermon)**: 확률론적 기반이 가장 탄탄하다. EBM, Score-Based 모델의 이론적 근거를 깊이 다룬다.
> - **MIT 6.S978 (Kaiming He)**: VAE부터 최신 모델까지 전체 계보를 다루며, 특히 DiT와 VAR/MAR 같은 최신 아키텍처에 강하다.
> - **MIT 6.S184 (Holderrieth)**: Flow Matching과 SDE/ODE 통합을 수학적으로 가장 깔끔하게 설명한다. 2026년 최신 강의.
> - **CMU 10-423 (Gormley)**: LLM과 생성 모델을 통합적으로 다루며, 실습 과제가 풍부하다.

13.2 추천 학습 경로

```
[현재 위치: 강의 1~5 + 이 보충 자료]
         |
         v
    ============================================
    |  Phase 1: 디퓨전의 핵심 (2~3주)             |
    |                                            |
    |  (1) DDPM 논문 정독 + 구현                   |
    |     -> 이 보충 자료 Section 3 수식 이해       |
    |                                            |
    |  (2) U-Net 구조 이해                         |
    |     -> Time Embedding, Skip Connection      |
    |                                            |
    |  (3) DDIM (빠른 샘플링)                      |
    |     -> ODE 관점 이해                         |
    ============================================
         |
         v
    ============================================
    |  Phase 2: 조건부 생성 (2주)                  |
    |                                            |
    |  (4) CLIP (텍스트-이미지 연결)                |
    |                                            |
    |  (5) Classifier-Free Guidance               |
    |     -> Section 6 수식 이해                   |
    |                                            |
    |  (6) Latent Diffusion / Stable Diffusion     |
    |     -> Section 5 파이프라인 전체 이해         |
    ============================================
         |
         v
    ============================================
    |  Phase 3: 최신 아키텍처 (2주)                |
    |                                            |
    |  (7) DiT (Diffusion Transformer)            |
    |     -> adaLN-Zero, 스케일링 법칙             |
    |                                            |
    |  (8) Flow Matching                          |
    |     -> Section 7 수식 이해                   |
    |     -> SD3, FLUX 구조 파악                   |
    |                                            |
    |  (9) SDE/ODE 통합 프레임워크                  |
    |     -> Section 8 (Yang Song 논문)            |
    ============================================
         |
         v
    ============================================
    |  Phase 4: 프론티어 연구 (2~3주)              |
    |                                            |
    |  (10) VAR / MAR (Visual AR 최신)            |
    |                                            |
    |  (11) Consistency Models (가속 생성)         |
    |                                            |
    |  (12) 비디오 생성 (Sora, Movie Gen)           |
    |                                            |
    |  (13) Diffusion Policy / World Models       |
    ============================================
```

> **각 Phase의 학습 전략 조언**:
>
> - **Phase 1**: 가장 중요한 단계. DDPM 논문(Ho et al., 2020)을 읽으면서 이 보충 자료의 Section 3과 대조하라. 수식을 직접 유도해보는 것이 핵심이다. 특히 $q(x_t|x_0)$의 closed-form 유도와 ELBO에서 Simple Loss로의 변환 과정을 종이에 직접 써보라.
> - **Phase 2**: CLIP과 CFG는 "텍스트로 이미지를 생성"하기 위한 **필수 구성요소**이다. LDM/SD의 파이프라인을 이해하면, 현재 사용되는 거의 모든 이미지 생성 시스템의 작동 원리를 파악할 수 있다.
> - **Phase 3**: Flow Matching은 최신 모델(SD3, FLUX)의 기반이므로, 향후 연구를 따라가려면 필수이다. SDE/ODE 통합은 이론적으로 깊지만, 다양한 모델 간의 연결을 이해하는 데 매우 유용하다.
> - **Phase 4**: 이 단계는 "현재 연구의 최전선"이다. 논문을 직접 읽으면서 이 보충 자료의 해당 섹션을 참고하라. 특히 Diffusion Policy와 World Models은 생성 모델의 미래 방향을 보여준다.

13.3 추천 학습 자료

자료	특징	난이도	링크
MIT 6.S184 Lecture Notes	Flow Matching + Diffusion을 수학적으로 가장 깔끔하게 설명	***	diffusion.csail.mit.edu
MIT 6.S978 (Kaiming He)	VAE -> AR -> GAN -> Diffusion 전체 + 최신 연구	****	mit-6s978.github.io
Stanford CS236	확률론적 기반 가장 탄탄, EBM 포함	****	deepgenerativemodels.github.io
Lilian Weng 블로그	Diffusion 개념을 가장 직관적으로 설명	**	lilianweng.github.io
Yang Song 블로그	Score-Based Model 원저자의 설명	***	yang-song.net/blog
CMU 10-423	LLM + 생성 모델 통합 시각, 실습 과제 풍부	***	cs.cmu.edu/~mgormley/courses/10423

> **자료 선택 팁**:
>
> - **처음 시작한다면**: Lilian Weng 블로그로 직관을 먼저 잡고, MIT 6.S184로 수학적 이해를 다지는 것을 추천한다.
> - **이론적 깊이를 원한다면**: Stanford CS236의 EBM, Score Matching 파트를 먼저 학습하라.
> - **최신 트렌드를 빠르게 따라가고 싶다면**: MIT 6.S978(Kaiming He)의 세미나 영상이 가장 최신 연구를 다룬다.

> - **구현 능력을 키우고 싶다면**: CMU 10-423의 과제를 직접 풀어보라. 또한 HuggingFace의 Diffusers 라이브러리 튜토리얼도 실습에 좋다.

부록: 핵심 수식 모음 (강의 1~5 + 보충)

A. 확률 분포 모델링의 4가지 접근

접근	$p(x)$ 관계	대표 모델	핵심 장단점
직접 모델링	$p_\theta(x)$ 자체를 학습	VAE, Normalizing Flow	우도(likelihood) 계산 가능, 모델 구조에 제약
암시적 모델링	샘플만 생성, p 불필요	GAN	고품질 샘플, 학습 불안정, 모드 붕괴
에너지 기반	$p_\theta(x) \propto \exp(-E_\theta(x))$	EBM	유연한 모델링, 정규화 상수 Z 계산 불가
스코어 기반	$\nabla_x \log p(x)$만 학습	Score-Based, DDPM, Flow Matching	Z 불필요, 안정적 학습, 느린 생성

> **이 네 가지 접근의 관계를 이해하면**: 이들은 "데이터 분포 $p_{\text{data}}(x)$를 어떻게 다룰 것인가"에 대한 서로 다른 답이다. 직접 모델링은 p를 명시적으로 구하고, GAN은 p를 몰라도 샘플만 잘 만들면 되고, EBM은 p의 unnormalized 버전만 학습하고, 스코어 기반은 p의 기울기(gradient)만 학습한다. 역사적으로 GAN이 먼저 고품질 이미지를 생성했지만, 스코어 기반/Diffusion 방법이 학습 안정성과 다양성에서 우위를 점하며 현재 주류가 되었다.

B. 전체 수식 총정리

VAE (강의 1):

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q(z|x)}[\log p(x|z)] + D_{KL}(q(z|x)\|p(z))$$

> 첫째 항은 **재구성 오차**(Reconstruction Error): 잠재 변수 z로부터 원본 x를 복원하는 능력. 둘째 항은 **KL 정규화**: 사후 분포 $q(z|x)$가 사전 분포 $p(z) = \mathcal{N}(0, I)$에 가까워지도록 강제.

GAN (강의 3):

$$\min_G \max_D \ \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$$

> 판별자 D는 진짜/가짜를 구분하려 하고, 생성자 G는 판별자를 속이려 한다. 이 미니맥스 게임의 **Nash 균형**에서 $p_G = p_{\text{data}}$.

VQ-GAN 2-Stage (강의 4):

$$\text{Stage 1}: \mathcal{L} = \|x - \hat{x}\|^2 + \|\text{sg}[z_e] - e\|^2 + \beta\|z_e - \text{sg}[e]\|^2 + \mathcal{L}_{\text{GAN}}$$

> $\text{sg}[\cdot]$는 stop-gradient (그래디언트 차단). 둘째 항은 코드북 업데이트, 셋째 항은 인코더 출력이 코드북에 가까워지도록 하는 commitment loss.

DALL-E (강의 5):

$$\mathcal{L} = -\sum_{i=1}^{M} \log p_\theta(t_i | t_{<i}) - \sum_{j=1}^{1024} \log p_\theta(s_j | t_{1:M}, s_{<j})$$

> 첫째 합은 텍스트 토큰의 자기회귀 손실, 둘째 합은 이미지 토큰(VQ 코드)의 자기회귀 손실. 텍스트와 이미지 토큰을 하나의 시퀀스로 연결(concatenate)하여 처리.

Score Function (강의 5):

$$s(x) = \nabla_x \log p(x), \quad s_{\text{Gaussian}} = -\frac{x-\mu}{\sigma^2}$$

> 스코어 함수는 로그 확률의 **기울기**: 데이터 밀도가 높은 쪽을 가리킨다. 가우시안의 경우, 평균 μ 방향을 가리키며, 분산 σ^2이 작을수록 더 강하게 끌어당긴다.

DDPM (보충 Section 3):

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon,\, t)\|^2\right]$$

> 핵심 직관: 노이즈 ϵ을 추가한 후, 신경망에게 "어떤 노이즈를 추가했는지" 맞추게 한다. 이것이 ELBO의 모든 복잡한 항을 하나의 간단한 MSE로 근사한 결과이다 (자세한 유도: Section 3.4).

CFG (보충 Section 6):

$$\tilde{\epsilon}_\theta = \epsilon_\theta(x_t, \varnothing) + w \cdot (\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \varnothing))$$

> 무조건 예측에서 조건부 예측 방향으로 **외삽(extrapolation)**한다. $w = 1$이면 표준 조건부 생성, $w > 1$이면 조건에 더 강하게 맞추되 다양성이 줄어든다.

Flow Matching (보충 Section 7):

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t,x_0,x_1}\left[\|v_\theta((1-t)x_0 + tx_1,\, t) - (x_1 - x_0)\|^2\right]$$

> 노이즈 x_0에서 데이터 x_1으로의 **직선 경로**를 따라, 속도장 v_θ가 상수 속도 $(x_1 - x_0)$를 예측하도록 학습한다. DDPM보다 경로가 직선에 가까워 더 적은 스텝으로 생성 가능.

Langevin Dynamics (강의 5):

$$x_{t+1} = x_t + \frac{\epsilon}{2}\nabla_x \log p(x_t) + \sqrt{\epsilon}\, z_t$$

> 스코어 함수(기울기)를 따라 올라가되, 노이즈 z_t를 추가하여 탐색(exploration)한다. 충분히 오래 반복하면 $p(x)$에서 샘플링하는 것과 동등해진다 (MCMC의 일종).

Consistency Model (보충 Section 11):

$$f_\theta(x_t, t) = f_\theta(x_{t'}, t') = x_0 \quad \forall t, t' \text{ on same ODE trajectory}$$

> ODE 궤적 위의 모든 점이 같은 x_0로 매핑된다는 **자기 일관성 성질**을 학습하여, 1~4 스텝만으로 생성 가능.

C. 생성 모델 계보도

```
                        [생성 모델의 역사]

 2013    2014    2015    2016    2017    2018    2019    2020    2021    20
  |       |       |       |       |       |       |       |       |       |
  |      VAE    PixelRNN         VQ-VAE                          DALL-E  LDM
  |       |       |               |                               |       |
  |       |    PixelCNN     VQ-VAE-2                         DALL-E 2    SDX
  |       |       |               |                               |       |
  |       +-------+------ 자기회귀 계열 --- GPT-2/3 ---- DALL-E ---+- Parti
  |               |
  |             NFlow ---- Glow ---- RealNVP           Normalizing Flow 계열
  |               |                                          |
  |               +---------------------------- CNF ----- Flow Matching --
  |                                                          |
  |      GAN -- DCGAN -- ProGAN -- StyleGAN -- StyleGAN2     |
  |       |                         |                        |
  |       |      CycleGAN    VQ-GAN --------------------- LDM에 통합
  |       |                                                  |
  |                                Score-Based -- DDPM --- DDIM -- DiT -- Sor
  |                                     |          |              |        |
  |                                     +---- SDE/ODE 통합 -- Consistency -- I
  |                                                 |
  |                                     EBM ---------+
  |
  +-- RBM / Deep Boltzmann Machine (역사적)
```

> **이 계보도를 읽는 방법**:
>
> - **세로 축**: 시간 (2013~2025)
> - **가로 연결**: 기술적 계승 관계 (화살표가 영향을 준 방향)
> - **핵심 합류점들**:
> - VQ-GAN + DDPM → **LDM/Stable Diffusion** (2022): 잠재 공간 + 디퓨전의 결합
> - DiT + Flow Matching → **SD3, FLUX** (2024-2025): Transformer 아키텍처 + 직선 경로
> - DDPM + Consistency → **가속 생성** (2023): 품질 유지하면서 1-4스텝
> - DiT + 비디오 → **Sora** (2024): 이미지 모델을 비디오로 확장
> - Diffusion + 로봇 → **Diffusion Policy** (2023): 생성 모델의 물리 세계 확장

이 보충 자료는 Stanford CS236 (Ermon, 2023), MIT 6.S978 (Kaiming He, Fall 2024), MIT 6.S184 (Holderrieth, IAP 2025/2026), CMU 10-423 (Gormley, 2024/2025) 커리큘럼을 참조하여 작성되었다.